uc3m | Universidad Carlos III de Madrid

Bachelor in Telematics Engineering

2024-2025

Bachelor Thesis

"Machine Learning-Based Predictive Modeling of Energy Prices"

Rodrigo De Lama Fernández

Emilio Parrado Hernández

Leganés, Spain, June 16th 2025

This work is licensed under Creative Commons **Attribution – Non Commercial – Non Derivatives**

SUMMARY

This project focuses on the development of a machine learning-based predictive model for electricity prices in Spain. Using historical data from the Iberian Energy Market Operator (OMIE) and technical analysis indicators, the model aims to accurately forecast hourly energy prices. The study focuses on a single hourly slot, evaluating the performance of various machine learning models, such as Linear Regression, Lasso, and Random Forest. The project employs a sliding window approach alongside feature engineering, employing technical analysis by calculating moving averages, momentum indicators, and volatility indicators. The results display the impact of using technical indicators as features to improve model accuracy and offer insights into the potential of data-driven approaches for energy price forecasting.

Keywords:

Energy, Machine Learning, Sliding Window, Technical Analysis (TA), Technical Indicators.

DEDICATION AND ACKNOWLEDGMENTS

Dedicated to my late grandfather, who was not able to see me become an engineer like himself. To my parents and my brother, that have always helped me push through hard moments. To my grandparents who will proudly see me become an engineer. To my amazing girlfriend that has supported me though all the ups and downs of life. To all of my friends and colleagues that have accompanied me these years. To anyone and everyone that has supported me during my years in university. Here is to the next steps in life.

I would like to acknowledge, and thank everyone that has helped me and supported me during the development of this project, highlighting the invaluable support of my tutor, my parents and my girlfriend. I would also like to extend a special thank you to all of the open source contributors of the libraries that have made this project possible.

CONTENTS

1. INTRODUCTION TO THE PROBLEM 1
2. BACKGROUND AND RELATED WORK. 3
2.1. Training a Non-Stationary Dataset with a Sliding Window Approach 4
2.2. Machine Learning Models Selection 4
2.3. Model Optimization: Hyperparameter Tuning and Feature Engineering 5
2.4. Evaluation Metrics 7
2.5. Related Work 8
3. PROPOSAL: SYSTEM DESCRIPTION 10
3.1. Prior Set Up and Configuration 11
3.2. Data Preparation: Ingestion and Cleanup 11
3.3. Feature Matrix Creation 13
3.4. Machine Learning Model Training 15
4. EXPERIMENTATION 18
4.1. Data Description 18
4.2. Model Set Up Explanation 20
4.3. Results of the Model Experiments 21
4.4. Discussion 29
5. REGULATORY FRAMEWORK 36
5.1. Data Availability 36
5.2. Software & Licenses 36
6. SOCIO-ECONOMIC ENVIRONMENT 38
6.1. Impact 38
6.2. Project Plan 38
6.3. Budget. 39
7. CONCLUSIONS 41
7.1. Revisiting the Objectives of the Research 41
7.2. Future Work 41
BIBLIOGRAPHY. 42

LIST OF FIGURES

3.1 Data Processing Pipeline 10

3.2 Simple Training Example 16

3.3 Simple Prediction Example 16

4.1 MarginalES Price Time Series at 14:00H 19

4.2 MarginalES at 14:00H Data Distribution, Including Mean, Median and Mode Values 20

4.3 Best Linear Regression Baseline Model 22

4.4 Best Linear Regression Feature Engineering Model 23

4.5 Best Lasso Baseline Model 24

4.6 Best Lasso Feature Engineering Model 25

4.7 Best Random Forest Baseline Model 26

4.8 Best Random Forest Feature Engineering Model 27

4.9 Baseline Linear Regression Metrics Distribution 29

4.10 Feature Engineering Linear Regression Metrics Distribution 30

4.11 Baseline Lasso Best Alpha Metrics Distribution 31

4.12 Feature Engineering Lasso Best Alpha Metrics Distribution 32

4.13 Baseline Random Forest Metrics Distribution 33

4.14 Baseline Random Forest Hyperparameter Analysis 33

4.15 Feature Engineering Random Forest Metrics Distribution 34

4.16 Feature Engineering Random Forest Hyperparameter Analysis 35

6.1 Gantt Diagram of the Project Lifecycle 39

LIST OF TABLES

4.1 Best Baseline Linear Regression Model Configurations 22
4.2 Best Feature Engineering Linear Regression Model Configurations 23
4.3 Best Baseline Lasso Model Configurations 24
4.4 Best Feature Engineering Lasso Model Configurations 25
4.5 Best Baseline Random Forest Configurations 26
4.6 Best Feature Engineering Random Forest Configurations 27
4.7 Best Performing Configuration for Each Model 28

6.1 Equipment Amortization 40
6.2 Human Costs 40
6.3 Final Cost Breakdown 40

1. INTRODUCTION TO THE PROBLEM

Energy plays an increasingly critical role in society, with reliable energy availability being essential for maintaining modern high-functioning societies across the world, that benefit from a robust and advanced energy infrastructure. Furthermore, electricity is a strategic resource needed to succeed in the ecological transition that we are currently undergoing world wide.

A important factor in this ever-growing necessity is the price of the available energy. But in recent times, that easy and affordable access has changed. Across Europe, it has become more unpredictable ever since the commencement of the war between Russia and Ukraine, which marked and inflection point in the mostly cyclical nature of electricity prices. Prices have gone up considerably in the past three years, and it has been affecting everyone. From big consumers such as companies, to individual consumers, we all now pay a more expensive electricity bill at the end of the month. This has become a topic of great concern for consumers, as they more often reflect upon the current situation.

These concerns led me to think about pricing, exploring how it was structured, and its predictability from the perspective of an end consumer. Can we plan our use of energy according to the price? Can the price be predicted accurately? These were some of the questions I had, which sparked my interest in looking into energy predictions.

After considering several modeling approaches, ranging from *Deep Learning* with long short-term memory (LSTM) neural networks, to statistical time-series specific models such as ARIMA, I chose to focus on traditional *Machine Learning* techniques. They offer a balance between predictive power and interpretability, building upon what was taught in the third year subject *Modern Theory of Detection and Estimation*. This decision was further supported by my advisor, Emilio Parrado Hernández, whose professional background in machine learning model design for investment banking shaped the direction of this work.

The core of this project revolves around an experimental system design that integrates the use of a sliding window training approach for traditional machine learning models, such as Linear Regression, Lasso, and Random Forest, with the use of expressive features created from financial indicators. This experimental use of technical analysis in the feature engineering phase is intended to investigate the effectiveness of these indicators applied in a different field. The idea is to aid the model training process by exposing it to different indicators that describe the trend, momentum and volatility of the time series.

En la intro tienes que explicitar los objetivos que persigues y un resumen del contenido de cada capítulo:

- Cap 2 background necessary to understand the project

- Cap 3 explains the design of the data processing pipeline

- Cap 4 will show the highlights of the results, and for the rest of the results, disccuss in depth the

- Cap 5 regulatory review

- Cap 6 socio-economic impact

Cap 7 conclusions an future work

2. BACKGROUND AND RELATED WORK

In Spain, the National Markets and Antitrust Commission, *Comisión Nacional de los Mercados y la Competencia (CNMC)* in Spanish, is the regulatory body in charge of regulating the functioning of the wholesale energy market, and the management and operations of the system. [1] Its methodologies must be followed by the designated energy market operator, the Iberian Energy Market Operator, *Operador del Mercado Ibérico de Energía (OMIE)*) in Spanish.

This was decreed on October 1st 2004, in the Santiago de Compostela International Agreement between the Kingdom of Spain and the Republic of Portugal. In article 4, section 2, it is agreed upon that the "OMIP will act as the managing entity of the forward market and OMIE as the managing entity of the day market, following the previous compliance with the rules of the Party in which territory the head-office is located." [2]

The OMIE publishes a guide that describes in depth the functioning of the daily energy market [3]. As the overseer of the market, they facilitate electricity transactions for the following day through offers from sellers and buyers. Every day at 12:00 CET, a market session determines electricity prices and energy volumes for the next day. Offers can be simple or complex, considering conditions like indivisibility, load gradient, minimum revenue, or scheduled shutdown.

The Iberian market has been coupled with the European market since 2014 for cohesion and integration purposes. Because of this, the OMIE's *Euphemia* algorithm is used to optimize economic surplus by matching supply and demand across different European markets, respecting market constraints, including interconnection capacity. After matching, results are sent to the System Operator, *Red Eléctrica de España (REE)* for validation to ensure technical feasibility. Adjustments may occur based on grid restrictions to ensure stability and efficient electricity distribution.

As a practical example, for a new producer to enter the daily energy market and sell electricity to the energy pool, they must take several key steps. The most essential one is registering as a producer for the Iberian electricity market, for which they will have to adhere to the current legal regulations and sign a contract to participate. They will then be able to submit offers to OMIE, which will result in increased competition. This increase in supply often leads to lower prices, specially if they introduce renewable energies, which have lower marginal costs compared to conventional energy sources such as nuclear or combined cycles.

2.1. Training a Non-Stationary Dataset with a Sliding Window Approach

Traditional machine learning models are usually trained on a fixed set of data, that is split into various subsets to ensure proper training and evaluation. Starting with the *training set*, which is the largest portion of the dataset and is used to train the model. The algorithm learns patterns, weights, and relationships within the data during this phase. The model's internal parameters are adjusted based on this set.

The *validation set* is used during model development to fine-tune hyperparameters and assess the model's performance on unseen data. It helps detect overfitting, which is when a model performs well on the training data but poorly on new data. The validation set guides model optimization choices without touching the final evaluation set.

The *test set* is strictly separate from the others. It is held out until the end of the training process, to be used to evaluate the final model's generalization capability. It provides an unbiased estimate of how the model will perform on truly unseen data, simulating real-world deployment conditions.

In the context of this project, the dataset is an energy price time series, which is not directly compatible with the standard training methodology, as over time the dataset characteristics might shift, making an arbitrary training set useless. This is because the considered time series is *non-stationary*.

The proposed solution to mitigate the *non-stationary* nature of the dataset, is to progressively change our training set. The way of doing this is by creating a *sliding window* of data points that will compose the training set. This sliding window will determine the number of past data points that are relevant for the next prediction, reducing the data set into one where we can assume it is locally *stationary*.

To work with this approach, the training set will be the entirety of the data points contained in the desired sliding window, and the test set will be the following row of data points (price values and features), which will be used to make the next prediction. Essentially, a new model will be created to make the following prediction, being discarded in favor of a new, slightly different model for the next prediction.

2.2. Machine Learning Models Selection

The project will explore the use of three models: Linear Regression, Lasso Regression, and the Random Forest Regressor.

Linear Regression [4] is one of the most fundamental and widely used statistical techniques for modeling the relationship between a dependent variable and one or more independent variables. The method assumes a linear relationship between the input features and the target, expressed as $y = X\beta + \varepsilon$, where β represents the model coefficients and ε is the error term. The model is trained by minimizing the residual sum of squares (RSS),

attempting to find the hyperplane that best fits the data. Due to its simplicity and interpretability, linear regression will serve as a baseline for our predictive modeling tasks.

Lasso Regression [5] (Least Absolute Shrinkage and Selection Operator) extends linear regression by introducing L1 regularization, meaning it penalizes the absolute magnitude of the regression coefficients. This penalty term, controlled by a hyperparameter *alpha* (α), encourages sparsity in the model by shrinking some coefficients down to zero, effectively performing feature selection. The optimization objective combines the ordinary least squares loss with the L1 penalty:

$$\hat{w} = \arg\min_{w} \left\{ \frac{1}{2n} \|y - Xw\|_2^2 + \alpha \|w\|_1 \right\}$$

This characteristic makes Lasso particularly useful in high-dimensional settings where many features may be irrelevant or redundant. While it can reduce overfitting and improve model generalization, it may also introduce bias by overly shrinking important coefficients, especially when predictors are highly correlated.

Finally, the *Random Forest Regressor* [6] is an ensemble learning method that constructs a collection of decision trees during training, and outputs the average prediction of the individual trees. Each tree in the "forest" is trained on a different sample of the data, and at each split, a random subset of features is considered, promoting diversity among the trees. This approach helps mitigate the overfitting commonly associated with single decision trees, and improves predictive accuracy and robustness. Unlike the previous models, Random Forest does not rely on assumptions of linearity or feature independence, making it well-suited for capturing complex, non-linear relationships in data. However, this flexibility comes at the cost of reduced interpretability compared to linear models.

2.3. Model Optimization: Hyperparameter Tuning and Feature Engineering

Hyperparameter tuning or *optimization* refers to the process of optimizing the model's training configuration parameters. These are parameters that are not learned from the data during the model training process, they are set before the training process begins and determine how the learning process of the model will develop. For *Lasso* Regression, *alpha* is a hyperparameter that controls the strength of the regularization. For the *Random Forest Regressor*, `n_estimators` (number of trees) and the `max_depth` (maximum depth of each tree), are a couple of it's hyperparameters. These values for any given model, and the dataset employed in its training, must be experimented in order to find the values that result in the best model performance (e.g., lowest error, highest accuracy) [7].

Feature Engineering is a crucial step in preparing, and expanding the dataset for training machine learning models. In this project, a particular approach involving *Feature Engineering with Technical Analysis* will be used, creating descriptive features based on the raw price data. Technical analysis, is a methodology for interpreting and analyzing

price movements using statistical data and historical patterns to make more accurate predictions [8]. This involves generating technical indicators such as moving averages and indicators for momentum or volatility.

These indicators are not just arbitrary transformations, they are intended to capture the inherent patterns and temporal information implicitly embedded inside the raw price series, theoretically reducing the need for complex time series models. The intention of generating these highly informative features, is to improve the robustness of the machine learning models, as the indicators should encode valuable predictive signals. This allows for a more straightforward model architecture while attempting to leverage the rich temporal dynamics of the data, as proven in other fields such as financial data analysis.

A variety of technical indicators such as *SMA*, *EMA*, *ROC*, *RSI*, *BBWidth* & *ATR* will be used. The *Simple Moving Average* (SMA) will be used with a variety of window sizes. A shorter SMA could be useful to identify short-term trends in energy prices, capturing recent price movements, while a longer SMA could be useful to capture long-term trends in energy prices, helping identify the overall direction of the market. The *Exponential Moving Average* (EMA) is similar to a simple moving average, but with the key difference being that it gives more weight to recent prices. This could be valuable to capture more immediate trends in energy prices. We will use the following *trend-following* indicators, where the sub indices represent the number of days considered in for the calculation of the indicator:

$SMA_3,\ SMA_5,\ SMA_7,\ SMA_{14},\ SMA_{30},\ SMA_{90},\ SMA_{180}$

$EMA_3,\ EMA_5,\ EMA_7,\ EMA_{14},\ EMA_{30}$

To measure momentum, the observed tendency of prices to continue moving in the same direction they have been moving in recently, the price *Rate of Change* (ROC) measures the percentage change between the current price and a price from a previous time period (e.g., 1 day, 7 days ago). It helps identify momentum in the market and is useful to highlight trends or sudden shifts in energy prices. Additionally, the *Relative Strength Index* (RSI) is another momentum indicator. It is mainly a financial indicator that can identify overbought (when a price has increased rapidly) or oversold (when a price has decreased rapidly) conditions in the financial markets. Regarding its usage in the energy markets, the intention is to track whether the price is approaching extreme values, which could indicate a reversal. We will use the following *momentum* indicators:

$ROC_3,\ ROC_5,\ ROC_7,\ ROC_{14},\ ROC_{30}$

$RSI_5,\ RSI_7,\ RSI_{14},\ RSI_{30}$

Finally, to measure volatility, the amount of price fluctuation or variation over time, we employ indicators such as the *Bollinger Band Width* (BBWidth) and the *Standard Deviation* (STD). We will use the following *volatility* indicators:

$BBWidth_7,\ BBWidth_{14}$

STD_7, STD_{14}, STD_{30}, STD_{90}

2.4. Evaluation Metrics

To evaluate the predictive performance of the models, a combination of statistical metrics that quantify how closely the predictions align with actual values will be employed. Evaluating using multiple indicators ensures a robust and comprehensive assessment, mitigating the risk of overfitting to a single metric, and providing a deeper understanding of both average-case and worst-case predictive behavior. This multi-metric evaluation framework is essential for iteratively refining the model and selecting the optimal sliding window, and set of hyperparameters.

The *Mean Absolute Error* (MAE) serves as a primary accuracy metric throughout the model development. It calculates the average absolute difference between predicted and actual values, in the same units as the target variable (€/MWh in this context). It offers a simple interpretation of how much, on average, the predictions deviate from reality. Since the MAE treats positive and negative errors equally, it provides a balanced view of model accuracy, which is particularly useful for the hyperparameters tuning process across iterations of the same sliding window. Its simplicity makes it ideal for tracking the performance of each parameter as the and the model training advances along the time series. The formula for the MAE is defined as:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^{n} \left| y_i - \hat{y}_i \right|$$

where y_i is the true value, $\hat{y}_i$ is the predicted value, and n is the total number of observations.

The *Mean Squared Error* (MSE) serves as a second core performance indicator. Unlike the MAE, the MSE penalizes larger errors more heavily, by squaring the differences between predicted and actual values, highlighting outliers or poor predictions. This characteristic makes the MSE particularly effective in identifying models that may perform well on average, but fail under certain conditions. It is especially useful in diagnosing issues related to overfitting or inadequate generalization. The formula for the MSE is defined as follows:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

The *coefficient of determination*, denoted as R^2, provides an intuitive measure of how well the model explains the variance in the target variable. An R^2 value close to 1 suggests that the model captures most of the variability in the data, while a value near 0 indicates a weak correlation between the input features and the target variable. This metric is particularly helpful when comparing multiple models or evaluating the effectiveness of adding additional data points with feature engineering, as it gives a high-level summary

of the overall model fit. The coefficient of determination (R^2) may be calculated with the following formula:

$$R^2 = 1 - \frac{\sum_{i=1}^{n} (y_i - \hat{y}_i)^2}{\sum_{i=1}^{n} (y_i - \bar{y})^2}$$

where $\bar{y}$ is the mean of the observed values.

Beyond average errors, the prediction accuracy can be also evaluated in terms of *percentiles* to understand how well the model performs across different segments of the distribution. For instance, checking the 75th, 85th, and 99th percentiles of error allows for an assessment of both typical and extreme prediction behaviors. This adds granularity to the evaluation process, highlighting whether the model is consistently accurate or only performs well under specific conditions. The percentiles will be calculated as the top performers in a given range, meaning only the example 75%, 85% or 99% of most accurate predictions would be considered.

Pivoting from the best percentiles, to the worst, a metric similar to *expectation shortfall* (ES), commonly used in financial risk analysis, was included as a measure of tail risk in model predictions. This self-generated ES-like metric will calculates the average prediction error among the worst-performing cases (e.g., the top 5% of errors), providing a quantitative estimate of the magnitude of extreme failures. In the context of energy prices, it helps us quantify how bad the prediction is when it fails, specially with over-estimations by hundreds of euros per MWh, since under-estimations shall be treated as 0, as we will work with the assumption that the price is not typically negative. This metric adds a risk-aware perspective to the model evaluation, ensuring that occasional large errors are accounted.

2.5. Related Work

In the specific context of European electricity markets, various studies have examined price prediction in countries such as Germany, Italy, France, and the United Kingdom, often taking into account market-specific features like renewable generation, cross-border interconnections, and regulatory frameworks. In parallel, several works focus on long-term forecasting horizons or on predicting renewable energy production, which indirectly affects market prices.

The present work is inspired by these contributions but follows a more targeted approach, focusing on short-term electricity price forecasting in the Spanish market using machine learning techniques and a rich set of technical indicators derived from historical price data.

While the prediction of energy consumption has been widely studied due to its relatively stable and cyclical nature, forecasting energy prices poses a significantly more complex challenge, as it is influenced by a broader set of volatile and interdependent factors such as market regulations, geopolitical events, renewable energy penetration, and

demand-supply imbalances.

Energy related predictions have

There are a variety of strategies to predict energy prices. There are various alternatives to machine learning, such as deep learning neural networks, or statistical time-series models. These are radically different approaches to the one taken in this project.

- Deep Learning for price predictions?

- LSTMs - Temporal Series?

Nogales et al. [9] proposed dynamic regression and transfer function models for next-day electricity price forecasting, applying their methodology to the Spanish and Californian markets.

- Energy consumption predictions?

3. PROPOSAL: SYSTEM DESCRIPTION

This chapter outlines the design of the system developed for the project, describing the complete *data processing pipeline* that was constructed. This will be explained starting from a blank state, detailing the complete set-up process required to run the project. Following this initial configuration, the data preparation stage will be outlined. This will be continued by the feature matrix creation process, concluding with the training of the machine learning models and the chosen evaluation method.

The following block diagram models the design of the system pipeline design in its most basic form:

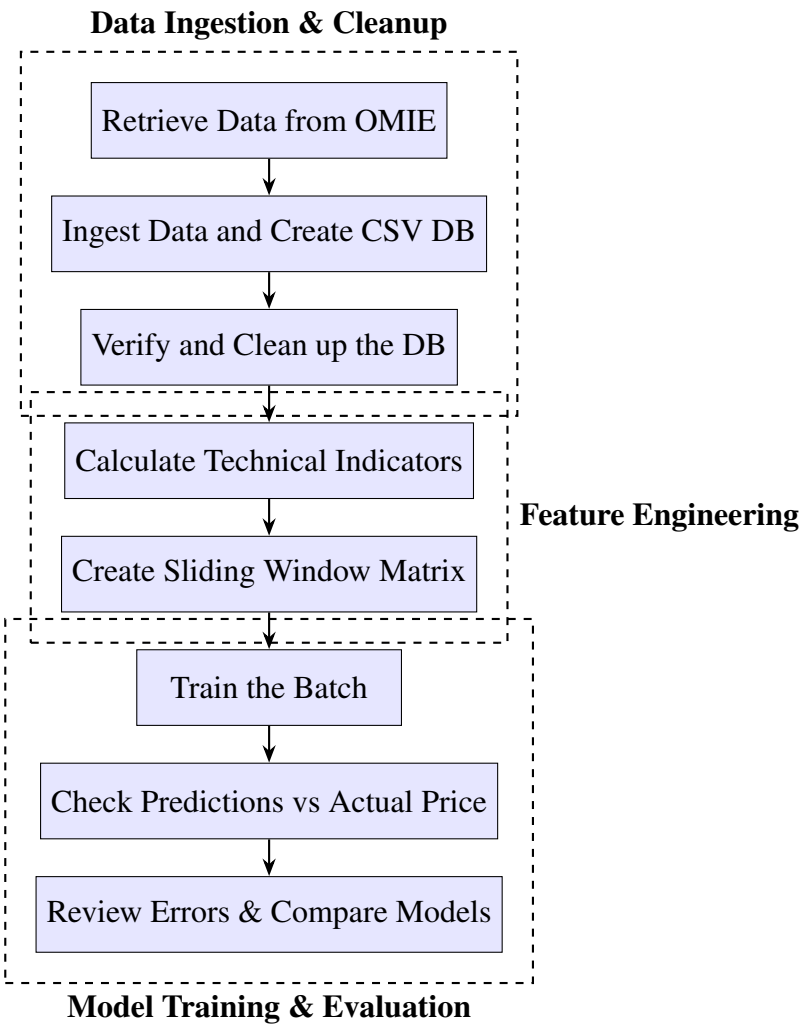

Fig. 3.1. Data Processing Pipeline

3.1. Prior Set Up and Configuration

Before getting into the system explanation, there are some important pre-requisites that must be met in order to run the project. The project was developed and tested with Python 3.13 [10], so for any recreation of the project, it is highly recommendable to use the same version. For additional assurance of reproducibility, and as a universal best practice for Python development, it is strongly advisable to set up a new virtual environment [11]. Alternatively, another system that would achieve the same goal of reproducibility, and dependency conflict avoidance would be the use of Miniconda [12]. This project uses the built in in Python module *venv*, with which a virtual environment can be easily created and configured with the project dependencies following the steps in *Appendix A*.

The chosen libraries were specifically selected to aid in the development of the project. Most of the project was written in Jupyter Notebooks, which are dependent on the *ipykernel* package. To aid in the retrieval of the data, the *requests* package was used to execute HTTP requests. For the manipulation of the data points, *pandas* was used to house the data structures, *numpy* for mathematical operations using arrays, and the *ta* technical analysis library was used to compute the technical indicators. Finally the machine learning algorithms were provided by the *scikit-learn* library.

Following the completion of this prior set up required for the project, the next sections will detail in depth the different steps involved in the system pipeline.

3.2. Data Preparation: Ingestion and Cleanup

The first step in any data preparation process would be to retrieve the raw data from the original source, in this case this means downloading it from the OMIE. It is publicly available data easily discoverable through their webpage [13]. The data collected was all that was initially available towards the end of 2024, with some extraordinary retrievals done later on in 2025 to further complete the set with more data points.

In this scenario, it meant retrieving data from 2018 onward. While for the 5 first years available the data was self contained in a compressed archive file, the rest from 2023 to our current 2025 was not. This meant manually downloading each daily data file. In order to streamline this process, a short Python script was created to build the necessary URIs to retrieve the files with an HTTP GET request.

The Python script in *Appendix B* was written with modularity in mind for ease of use. It firstly creates the filenames according to the OMIE's file naming convention, inserting them into a list. This list is essential to successfully build the URIs with which the HTTP GET request will be made. Finally the function `download_files(urls)` makes the request, retrieving the files.

The newly downloaded raw data was manually organized into its corresponding folder per year. An example filename is the following: `marginalpdbc_20230102.1`. This is an

important organizational step for the ingestion of the data, as the next procedure involves a script that iterates through these folders, identifying all files that start with `marginalpdbc_`, and parsing them into an easier to use form, a Pandas DataFrame [14].

When a valid file is identified while iterating through the *year* folders, the script reads its content. It will perform some cleanup steps, such as removing the first and last rows which are a standard header and footer across all files. It then parses the remaining data, which is separated by semicolons, into a Pandas DataFrame for that specific day. It will name the columns to *Year*, *Month*, *Day*, *Hour*, *MarginalPT* (Portuguese Energy Price), and *MarginalES* (Spanish Energy Price). It will also filter out any rows where the *Year* column is not a number, ensuring data integrity. The columns are then converted to their appropriate data types, *integers* for date/time components and floats for marginal prices. Each processed daily dataset is then temporarily stored in a list.

After processing all the individual files, the script concatenates all the daily data from the list of DataFrames into a single, comprehensive DataFrame. It will then construct a *Datetime* column built from the *Year*, *Month*, *Day*, and *Hour* columns. It will also handle some peculiar cases where an *Hour* value of 25 might appear (likely due to daylight savings time zone adjustments) by converting it to 0. This *Datetime* column is then set as the DataFrame's index. Finally, it removes the individual date and hour columns as well as the *MarginalPT* column, leaving only the *MarginalES* as the primary target feature, along with the *Datetime* index.

This clean and combined dataset will be saved as a CSV file for ease of use, `raw_data.csv`. The script then performs further data cleanup by reloading the `raw_data.csv`, and sorting it by *Datetime*, saving it as `processed_data.csv`. The whole process concludes by verifying the completeness of the hourly timestamps within the processed data, generating the full range of hourly timestamps that should be there, and checking for any discrepancies. If no missing timestamps are found, it confirms successful data processing; otherwise, it reports the missing timestamps, indicating an error in the data collection or processing.

For the previous in-depth explanation, you may find in *Appendix C* the of Python script that was created for the desired data ingestion and cleanup tasks.

After completing this ingestion and cleanup process, the dataset will be reduced to just the 14th hour data point. The code in *Appendix D* will create a the final database file, which will house the singular *Datetime* value, and the *MarginalES* data point.

To aid in the predictions, the dataset is enhanced with more features. We can calculate a variety of different features to aid the training of the model. These can range from simple variables such as the day of the week, the month of the year, or if that day is a weekend or not, to more complex, such as technical indicators. By adding these auxiliary features derived from *technical analysis*, we can further expand the context that our models will be able to learn from, ideally improving their prediction accuracy capabilities.

To generate these additional features, and the strategically chosen technical indicators, the main database `processed_data.csv` is read into a DataFrame that will be progressively expanded with new feature columns. The resulting DataFrame will be saved to a secondary database `ta_metrics_hour_14.csv`. The code in *Appendix E* is an example of the process that was employed to calculate the different indicators, using the open source Python library *ta* [15].

3.3. Feature Matrix Creation

The creation of a *feature matrix* is a key step in the process of training any machine learning model. In this context, the objective is to structure historical energy price data into a form that can be used to effectively train models to predict future prices. The correct transformation of the raw time-series to form our feature matrix is critical for this process. The intent of building the feature matrix consists on creating one unique matrix that can be used across any desired model. The feature matrix denoted as X, represents the input data used to train the model, while the target / label vector y contains the values the model is intended to learn how to predict.

The simple feature matrix of an example time series of 6 price values x_1 through to x_6 can be represented as:

$$X = \begin{bmatrix} x_1 & x_2 & x_3 \\ x_2 & x_3 & x_4 \\ x_3 & x_4 & x_5 \end{bmatrix}, \quad y = \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix}$$

Where each row of X corresponds to a sliding window of past values, and its corresponding label y is the value that follows that window, which would be the actual energy price for the current day:

$$\begin{aligned} y_1 &= x_4 \\ y_2 &= x_5 \\ y_3 &= x_6 \end{aligned}$$

This setup reflects the temporal dependency in the time-series data, where future values are predicted based on a certain number of past observations. This approach allows the machine learning models to "learn" these implicit patterns in the considered sliding window, enabling them to better predict the next value.

Incorporating technical analysis indicators into the feature matrix introduces additional information such as moving averages or the rate of change calculated over historical price windows, These enrich the model's inputs and are added as extra columns to the feature matrix:

$$X = \begin{bmatrix} x_1 & x_2 & x_3 & ta_{11} & ta_{21} & \cdots & ta_{n1} \\ x_2 & x_3 & x_4 & ta_{12} & ta_{22} & \cdots & ta_{n2} \\ x_3 & x_4 & x_5 & ta_{13} & ta_{23} & \cdots & ta_{n3} \end{bmatrix}$$

Where:

- x_i are raw price values from previous time steps,
- n is the number of indicators,
- ta_{ji} is the value of the j-th technical indicator computed based on the last price input of row i, i.e., x_{i+2} in this 3 previous days example.

The target / label vector will remain unchanged:

$$\mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix}$$

This design is motivated by the nature of the energy market data, which is *non-stationary*, as the statistical properties of electricity prices (e.g. mean, variance) change over time due to evolving supply and demand dynamics, regulatory interventions, and renewable energy contributions. Since the dataset has inherent *temporal dependencies*, as prices are typically dependent on recent values, making the assumption of independent and identically distributed (i.i.d.) samples invalid. The aim is to identify *local trends*, implementing a sliding windows approach allows the model to capture short-term patterns and trends without requiring the data to be stationary.

By using a sliding window mechanism, the models will be trained on a continuous stream of overlapping data segments. This not only increases the number of training samples, enhancing generalization, but also aligning with the real-world scenario of forecasting the next value based on a recent history of observations. The ultimate aim is to build the feature matrix using the sliding window approach to provide a robust and flexible representation of temporal data, allowing traditional machine learning models to operate more effectively in a domain typically reserved for time-series-specific methods.

The Python Script found in *Appendix F* contains the code necessary to create the simple feature matrix, which was initially generated based off *MarginalES* values, with the desired sliding window width for that training batch. For an example run we can retrieve from the database a sliding window of 6 days:

```
                Datetime  MarginalES
187 2019-01-01 14:00:00       65.88
188 2019-01-02 14:00:00       63.16
189 2019-01-03 14:00:00       66.70
190 2019-01-04 14:00:00       69.17
191 2019-01-05 14:00:00       64.00
192 2019-01-06 14:00:00       64.86
```

For which the resulting simple feature matrix is the following:

```
print(X_simple)
0      1      2
0  65.88  63.16  66.70
1  63.16  66.70  69.17
2  66.70  69.17  64.00

print(y_simple)
0    69.17
1    64.00
2    64.86
```

The simple feature matrix function was later modified and extended to incorporate additional feature columns such as the focus of our proposal, the technical indicators. These are the previously calculated features in order to expand the context our models will be able to train with. The script located in *Appendix G* demonstrates the flexible and auto-adaptable nature code, suitable for any number of inputted features (columns) obtained from the secondary feature enhanced database, which includes the additional technical indicators.

Furthermore, for an example run of the matrix creation with the technical indicators we can retrieve the extended database and select the same date range as in the previous example for comparison. Where we may observe an identical matrix to the earlier one, extended laterally to include the new feature columns, such as the technical indicators and additional *"Datetime"* related context:

```
print(X_extended)
   price_t-3  price_t-2  price_t-1      SMA_3   SMA_5      SMA_7     SMA_14  \
0      65.88      63.16      66.70  65.246667  64.974  63.842857  64.975000
1      63.16      66.70      69.17  66.343333  66.026  64.490000  65.401429
2      66.70      69.17      64.00  66.623333  65.782  65.434286  65.330000

      SMA_30     SMA_90    SMA_180  ...     RSI_30  BB_Width_7  BB_Width_14  \
0  64.886000  65.044333  67.227333  ...  50.823486   17.871701    15.638302
1  64.988333  65.038667  67.272889  ...  52.153121   21.198596    16.528083
2  64.947333  65.184222  67.267611  ...  49.268675   11.634263    16.685668

      STD_7    STD_14    STD_30    STD_90  month  day_of_week  is_weekend
0  3.080999  2.636139  2.620006  4.626188    1.0          3.0         0.0
1  3.691585  2.804414  2.726835  4.620755    1.0          4.0         0.0
2  2.055690  2.828060  2.732317  4.369895    1.0          5.0         1.0

print(y_extended)
0    69.17
1    64.00
2    64.86
```

3.4. Machine Learning Model Training

There are multiple strategies that can be implemented for an effective model training, depending on if the dataset is *stationary* or not. An *incremental* approach can be taken when de data is *stationary*, since in this case, the accumulating the data points will benefit the model's training. An *adaptive* approach is in order when the dataset is of *non-stationary* nature. Similar to an exponential moving average, where the newer data holds more weight, while the older data's importance is progressively minimized, this investigation's models were trained with a sliding window approach, where the older data points are cut off. This training method "forgets" the earliest values as they are no longer relevant in order to make the latest prediction, which is is the case in the energy prices database.

The idea behind the previously introduced *adaptive learning* approach is simple, we will create a 1 day step loop across the complete time series. A finite set of days will be used in training each model, with each instance being utilized for one single prediction. The training phase will consist on searching the optimal sliding window of days, with the calculated set of features. In each iteration, the model will be trained with a set feature matrix, and will then be fed with the next feature row to test its prediction abilities.

Continuing with our previous example, the model will be trained with the feature matrix X.

$$X_train = \begin{bmatrix} x_1 & x_2 & x_3 \\ x_2 & x_3 & x_4 \\ x_3 & x_4 & x_5 \end{bmatrix}, \quad y_train = \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix} \longrightarrow \boxed{\text{ML Model Training}}$$

Fig. 3.2. Simple Training Example

In order to predict a new value, for example, $\hat{y_4}$, the model will be fed the next row of feature values $\{x_4, x_5, x_6\}$, where x_6 will be our last known value, y_3.

$$X_to_predict = \begin{bmatrix} x_4 & x_5 & x_6 \end{bmatrix} \longrightarrow \boxed{\text{ML Model Predict}} \longrightarrow \hat{y_4}$$

Fig. 3.3. Simple Prediction Example

The model selection for the project will consist of three models: Linear Regression, Lasso Regression, and the Random Forest Regressor. The first of the models, *Linear Regression* [4] will serve as our baseline. This model assumes that the target variable can be approximated as a linear combination of the input features. With the sliding window approach it will be possible to test multiple configurations of the model, but in comparison to the other tested models, it does not have an explicitly tunable hyperparameter.

Our second model, the *Lasso Regression* [5], extends the linear model by introducing *L1 regularization*, meaning it has the ability to penalize unused features. This penalty encourages sparsity in the feature weights, effectively performing a feature selection. By tuning its hyperparameter *alpha* (α), we can modify the strength of the regularization. This can potentially improve the model's performance, by reducing overfitting, possibly improving its generalization capabilities. To optimize its predictive accuracy, we will later experiment across a variety of *alpha* hyperparameter values.

The third and final model considered in this research project, is the *Random Forest Regressor* [6], an ensemble learning method based on decision trees. During training, it constructs multiple decision trees, and aggregates their outputs by averaging the regression tasks. We can tune its training process by adjusting parameters such as `n_estimators` (number of trees) and the `max_depth` (maximum depth of each tree). Its output will be averaged, which can be a limiting factor for its predictions, since even if it is capable to learn more complex patterns than the aforementioned models, it will tend to underestimate extreme peak values. This model has been included as a non-linear alternative to validate the performance of the linear models and capture more complex relationships within the data.

Regardless of the chosen model, the training strategy will be as follows: After importing the complete dataset, we will define the number of days you want to include in the sliding window. This determines how many days from the time series will be included in the training. At the same point, we will select the desired number of previous days to be included in the feature matrix as columns. Depending on the selection of the sliding window size, there will be more or less models to train.

After initializing the variables where the predictions, actual values, and their time stamps will be stored at, the training loop will begin. Here, a new model will be trained per iteration as seen in Figure 3.2, and a new value will be predicted as seen in Figure 3.3. Once all models and predictions have been processed, the `prediction_df` DataFrame will be available for evaluation of the predictive precision of the training loop.

To finalize the system description, the example script found in *Appendix H* shows in Python code the basic function definition for our sliding window training process, in which any given model could be used. However, it is a basic example that does not test multiple sliding windows, nor consider any hyperparameter tuning. This will be covered in the following section, in the dedicated *Experimentation* chapter.

4. EXPERIMENTATION

This chapter presents all of the relevant information pertaining the experimentation phase of the project. We review the specifics of the data that was dealt with, and explain the decisions taken for the preparation of such data, and the training of the different machine learning models employed. We review all of the results of the different models, and discuss in depth the different set-up variations for the models.

4.1. Data Description

The data that was used for this project was exclusively publicly available information, published by the OMIE. The data retrieved from their website [13] for this project takes the following shape:

```
MARGINALPDBC;
2018;01;01;1;28.1;6.74;
2018;01;01;2;33;4.74;
2018;01;01;3;32.9;3.66;
...
2018;01;01;22;28.1;21.95;
2018;01;01;23;28.1;23.52;
2018;01;01;24;27.6;16.35;
*
```

We can understand and interpret this format following the OMIE's guide: *Modelo de Ficheros para la distribución pública de Información del mercado de electricidad 1.35* [16]. In page number 67, chapter 6.18 we may find the following information regarding the encoding format for the information:

```
6.18 Precios marginales del mercado diario (MARGINALPDBC)

Fichero con los precios marginales del Mercado Diario para cada una de
las horas.
Nombre del fichero: marginalpdbc_aaaammdd.v donde aaaammdd
corresponde a la fecha de sesión y v es la versión del fichero.

Descripción de los campos:
CAMPO DESCRIPCIÓN VALORES VÁLIDOS
Año Año I4 - 20XX
Mes Mes I2 - 1 a 12
Día Día I2 - 1 a 31
Hora Hora I2 - 1 a 25
```

```
MarginalPT Precio marginal zona Portuguesa F8.2 - -99999.99 a 99999.99
MarginalES Precio marginal zona Española F8.2 - -99999.99 a 99999.99
```

From this data description we can obtain the following column names: *Año, Mes, Día, Hora, MarginalPT* and *MarginalES*. As a proof of concept, in this research project we will be focusing on a single time slot, simplifying the data, from multiple data points per day, to a single one. We are exclusively interested in the value of the last column, *MarginalES*, and specifically the row pertaining the 14th hour of each day.

Dataset Analysis and Interpretation

After the data is ingested into the database, we can inspect the time series by visualizing the data points, and the data distribution, including the mean, median, and mode values of the dataset.

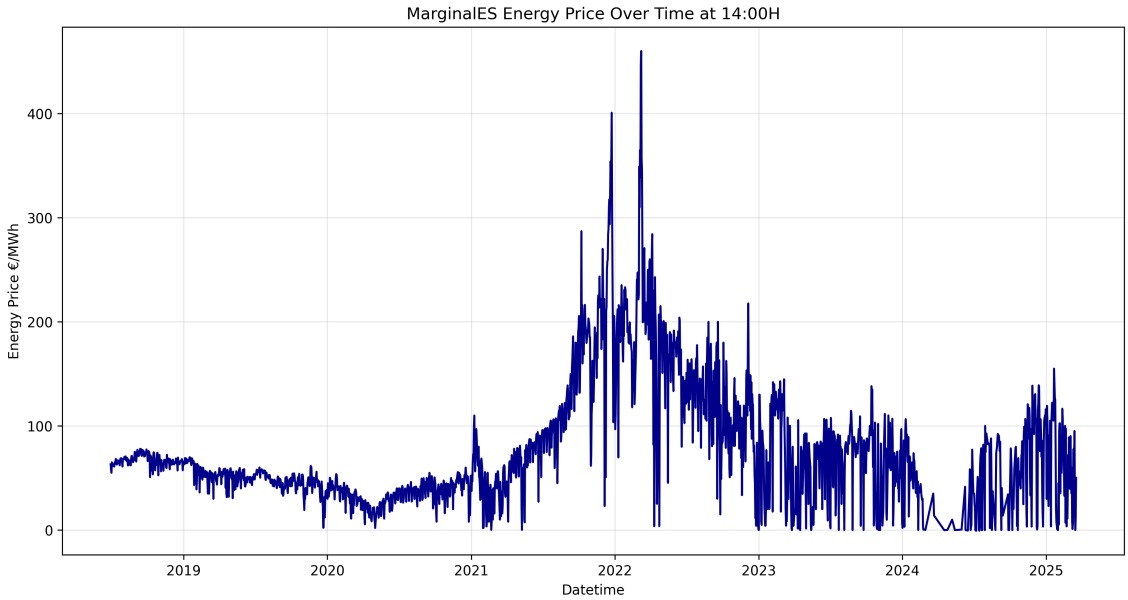

Fig. 4.1. MarginalES Price Time Series at 14:00H

A visual inspection of the dataset reveals that the observations that form the time series does not meet the assumptions of being independently and identically distributed (i.i.d.) and is also clearly non-stationary. The series exhibits temporal dependencies, with strong correlations between consecutive observations. The aforementioned *sliding window* approach will be the key to correctly processing this time series with techniques usually meant for working with i.i.d. datasets,

The energy prices show distinct periods, starting with relatively stable prices around 50-70 €/MWh from 2018 to late 2020, followed by growing volatility and a dramatic surge beginning in late 2021 that peaks at 460 €/MWh in 2022. This price escalation coincides with the escalating tensions, and eventual start of the war between Russia and Ukraine in February 2022, which severely disrupted European natural gas supplies and created unprecedented energy market volatility.

Since natural gas is a crucial input for electricity generation, this geopolitical crisis directly impacted electricity prices by disrupting the matching of supply and demand in OMIE's daily

market, where gas-fired power plants often set the marginal price. The subsequent gradual decline in 2023-2024 reflects market adaptation with alternative supply arrangements, and a reduced gas dependency, though prices remain elevated compared to pre-war levels.

These clear shifts in behavior over time show that the data is not *stationary*, as the average values, variability, and overall distribution change across different periods. Because of this, it makes sense to use an *adaptive* sliding window approach, where the model is regularly retrained on the most recent data. This helps it stay up to date with the latest trends and better handle the ups and downs of a changing energy market.

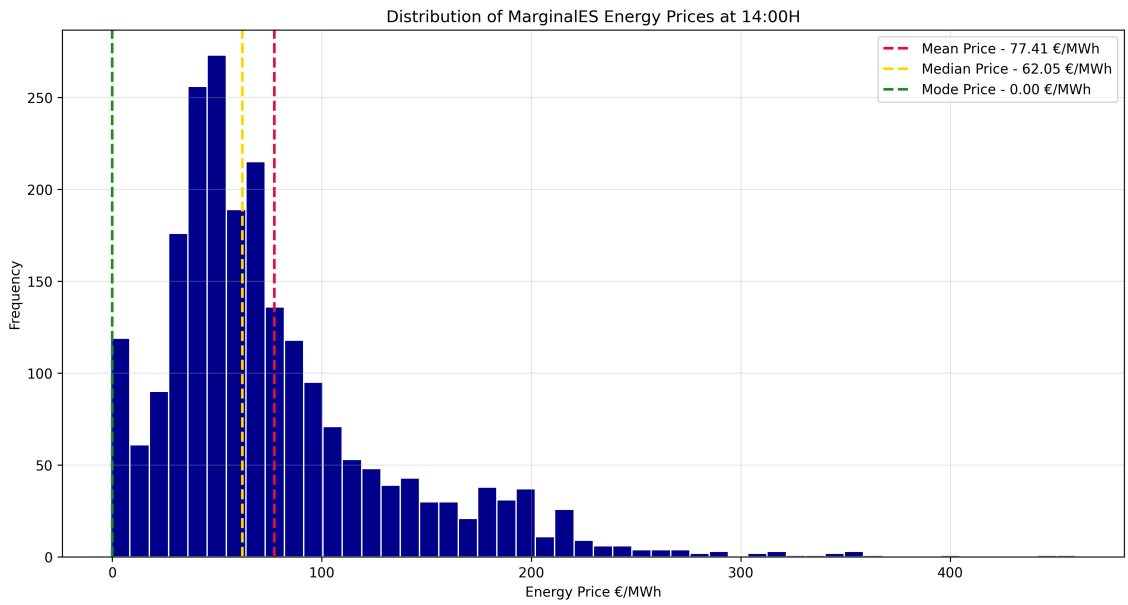

Fig. 4.2. MarginalES at 14:00H Data Distribution, Including Mean, Median and Mode Values

An interesting observation to comment upon is the mode value, which as seen in the graph, is zero. This can be attributed to the selected hour of the day for our dataset being 14:00H, which is usually around the time when the sun will be at its highest point in the sky, marking the peak production time for solar photovoltaic energy in Spain. And since Spain has an extremely large amount of solar power plants, this means that there will be an overabundance of energy, and therefore the price will plummet down to 0.

In general, the time series posses a high correlation between the prices of neighboring days, with seasonal variations, slow and progressively changing the price. The task at hand for the models is to figure out the best fit to the time series for any of the proposed sliding window configurations.

4.2. Model Set Up Explanation

To assess the predictive performance of the different models introduced earlier, a series of controlled experiments were implemented. In order to successfully run these experiments, the

previously explained model training was modified and extended. The most significant modification was the addition of support for batch training across multiple *temporal window configurations*, referring to the combination of the key parameters common to all models: `sliding_window` and `lag_price_window`. These parameters are crucial to identify the best performing temporal windows for each model's training.

For each model, a baseline version was first trained and tested exclusively using the raw time series data. This will serve as the primary comparison benchmark to evaluate the added value of the feature engineering process across each of the models.

For *linear regression*, the focus was solely on evaluating its baseline performance against the engineered features. This was done across a variety of temporal window configurations, without any further hyperparameter adjustments, since linear regression does not include tunable parameters.

The *lasso regression* experiments expanded upon the different temporal window configuration training batches, by also testing across multiple values of its regularization hyperparameter *alpha*. In this model, the objective was to assess its impact on feature selection and model generalization.

For our final model, a similar was done for the *Random Forest*, not only testing across multiple temporal window configurations, but also exploring different combinations of tree depth (`max_depth`) and the number of leaf nodes (`n_estimators`) to control model complexity and prevent overfitting.

The following sections will present the specific configurations and numerical values used for each of the tested parameters across the different models. This includes the temporal window parameters, as well as the hyperparameters specific to each model. The experiments and their performance evaluations presented in the next sections are entirely based on the models trained using these specific parameter configurations.

4.3. Results of the Model Experiments

This section presents the best results obtained across the different models. It provides a general overview of the performance potential of each of the top model configurations, analyzing the impact of both feature engineering and hyperparameter tuning on the final outcomes. All of the following results presented correspond to the 99th percentile of prediction errors, meaning the top 1% of the largest errors were excluded. This was chosen in order to fairly represent performance while minimizing the influence of extreme outlier predictions that are caused by isolated model errors.

The considerations taken to decide the top performing model were simple and focused on practical predictive performance. A custom score was created that averages out the ranking of the 4 calculated metrics, the Minimum Average Error (MAE), Mean Squared Error (MSE), the coefficient of determination (R^2), and our metric similar to Expectation Shortfall (ES-like), which in our experiments will account for the top 5% of errors inside the considered 99th percentile. The MAE, MSE and ES-like metric are ranked by ascending order, meaning the lower the metric, the higher the rank. In contrast, R^2 is ranked by descending order as its maximum value is 1, so the closest to it will have the highest the rank.

Note: An example error and accuracy metric calculation process is available to review in *Appendix I*.

Note: For direct comparison to the other metrics, all MSE metrics will be presented as its root, RMSE, which does share the same units (€/MWh).

Linear Regression

Starting with the testing, the first model used will be linear regression. It does not have any tunable hyperparameters, so no manual model optimization was done. The training batches were limited to testing different sliding windows.

The first experiment developed was a baseline training run, limited to just the the raw time series, without any feature engineering. The results of the top three model configurations, and the plot for the best performer are as follows:

Sliding Window	Lag Window	MAE (€/MWh)	RMSE (€/MWh)	ES-like (€/MWh)	R^2
30	1	15.195	22.798	66.472	0.842
30	2	15.472	23.039	66.299	0.841
90	1	15.817	23.340	67.367	0.838

TABLE 4.1. BEST BASELINE LINEAR REGRESSION MODEL CONFIGURATIONS

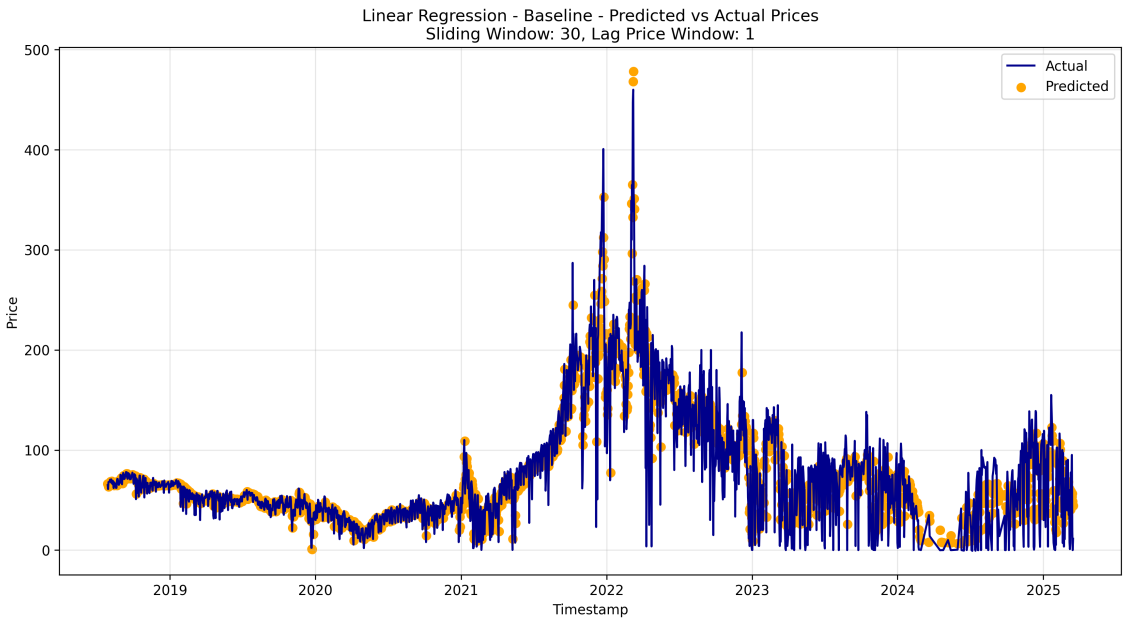

Fig. 4.3. Best Linear Regression Baseline Model

A visual inspection indicates that the model correctly tracks the time series, performs reasonably well during periods of relatively stable prices (2018-2020 and parts of 2023-2024), where

predicted values closely follow actual prices. We can observe that the model underperforms during the major price spikes in 2022, suggesting that it struggles with volatility. At times, there also appears so be some lag in predictions, especially during rapid price changes, where the model appears to react to trends rather than anticipate them. This is potentially due to the 30-day long window, and short 1-day lag price, that might not give enough context. As a baseline model, this simple approach provides a good starting point for comparison.

The next experiment consists of using the more complex feature matrix, which includes the calculated technical indicators. This will review the potential improvements and variations of the feature engineering training run, in comparison to the baseline run, while recognizing the trade-off between capturing additional underlying patterns and introducing noise or redundancy that can lead to overfitting in linear regression. The temporal windows that will be tested are: sliding_windows = [7, 10, 15, 30, 90] and lag_price_windows = [1, 2, 4, 6]. The best feature engineering results, and the plot of its best performer are the following:

Sliding Window	**Lag Window**	**MAE (€/MWh)**	**RMSE (€/MWh)**	**ES-like (€/MWh)**	**R^2**
7	6	15.062	22.798	75.703	0.819
90	1	22.915	23.039	113.731	0.616
90	2	23.366	23.340	116.260	0.601

TABLE 4.2. BEST FEATURE ENGINEERING LINEAR REGRESSION MODEL CONFIGURATIONS

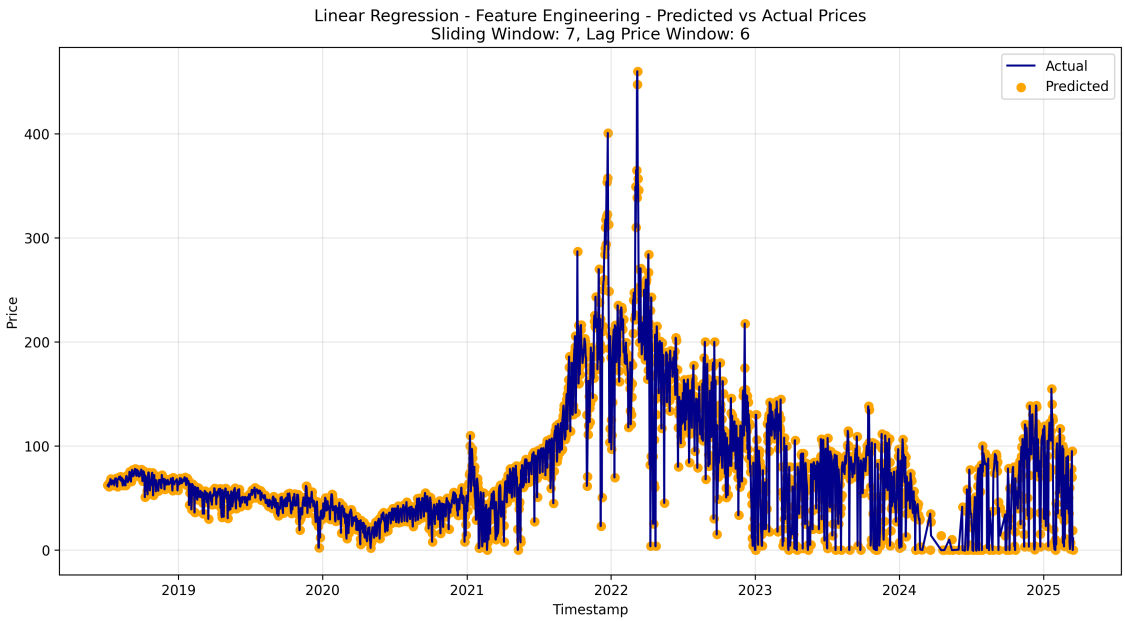

Fig. 4.4. Best Linear Regression Feature Engineering Model

From the results we can observe a clear improvement when dealing with price surges during volatile market conditions. The predictions appear to respond more quickly to price changes, with

less lag compared to the baseline model. This behavior can be noticed in the 2021 price rise, where the baseline model tended to underestimate values, while the feature engineered one is captured well the day to day variations, including local peaks. In general there are consistently fewer over or under-estimations across the whole time series.

Lasso Regression

For second model considered in this evaluation, Lasso regression is also a linear model that penalizes unused features. It does have a tunable hyperparameter *alpha*, enabling manual model optimization. A selection of parameter values will be tested in every sliding window iteration, noting down the *alpha* value that renders the result with the minimum error to the actual data point. In addition to all the previous temporal window configurations, the following *alpha* values were tested: alphas = [0.01, 0.1, 1, 10, 100, 1000]. The best baseline model configurations, and the top performer plot are the following:

Sliding Window	Lag Window	Alpha	MAE (€/MWh)	RMSE (€/MWh)	ES-like (€/MWh)	R^2
30	1	0.01	15.196	22.798	66.472	0.842
30	1	0.10	15.197	22.799	66.472	0.841
30	1	1	15.213	22.803	66.469	0.838

TABLE 4.3. BEST BASELINE LASSO MODEL CONFIGURATIONS

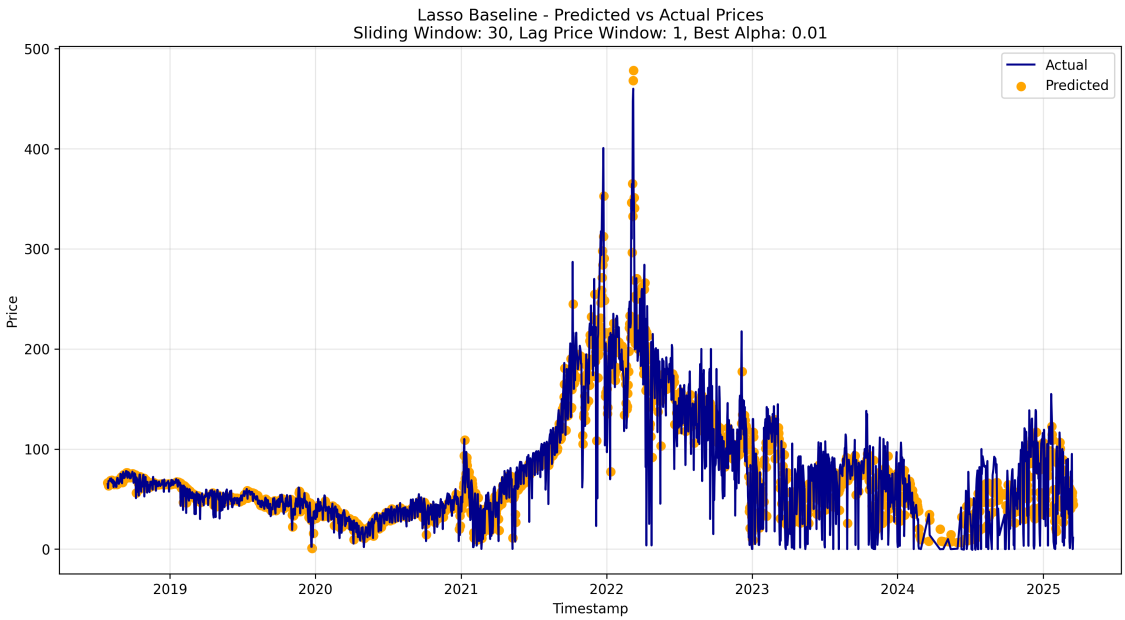

Fig. 4.5. Best Lasso Baseline Model

From the results from this training we can observe that the predictions are almost identical to the linear regression baseline. This is due to the best performant Lasso model being attained with the lowest *alpha* hyperparameter (0.01), meaning it is not performing any feature selection, effectively working as the previous linear regression model.

As with the previous baseline model, we will explore the effect of expanding the dataset with technical indicators. The feature engineering training batch results, and the plot of the best performer are the following:

Sliding Window	Lag Window	Alpha	MAE (€/MWh)	RMSE (€/MWh)	ES-like (€/MWh)	R^2
7	6	0.01	15.062	24.351	75.703	0.819
7	6	10	15.062	24.351	75.703	0.819
7	6	1000	15.062	24.351	75.703	0.819

TABLE 4.4. BEST FEATURE ENGINEERING LASSO MODEL CONFIGURATIONS

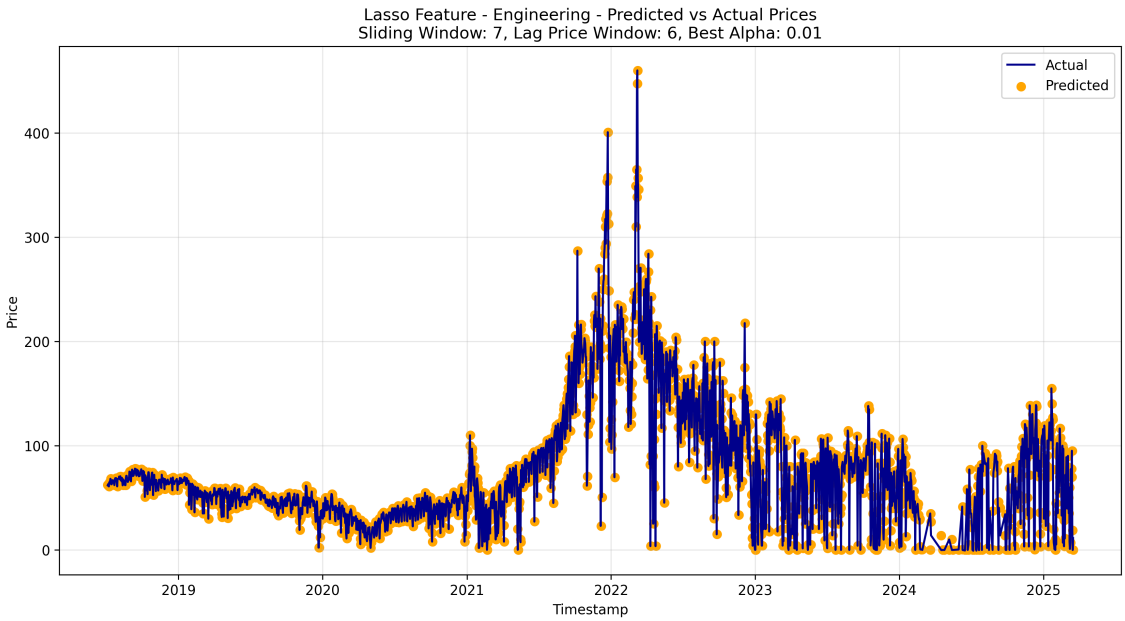

Fig. 4.6. Best Lasso Feature Engineering Model

This final linear plot reveal several interesting points about combining regularization and feature selection. We can visually confirm that once again, the results are identical to the linear regression feature engineering results using the same model configuration. An interesting observation to make is that the results were not alpha value dependent, all of the three plots looked identical. This suggests that both the temporal window configuration and the feature selection are well optimized, making the regularization process less critical.

These results can be interpreted as a positive finding, suggesting that the feature engineering procedure has created a stable and optimal feature set that doesn't require fine-tuning of the regularization parameter. Since the model is robust across different alpha values, it will reliably perform since the selected features are truly important.

This demonstrates that well-engineered features can make regularization almost irrelevant.

With the right temporal features, even simple linear models can achieve good performance in electricity price forecasting.

Random Forest

The last model that was experimented with was the Random Forest regressor. The two main parameters that we will focus on tuning are the number of trees in the forest (`n_estimators`) and the maximum depth of the tree branches (`max_depth`). Similar to the training process of Lasso regression, for every sliding window iteration each combination of parameters will be tested. Likewise, we will save the combination of parameters with the best result, with the least error to the actual data point. All of the previous temporal windows will be considered, alongside n_estimators = [100, 200, 300, 400] and max_depths = [None, 10, 20, 30]. The results for the best baseline runs, and the plot of the best performer are the following:

SW[a]	LW[b]	n_estimators	max_depth	MAE (€/MWh)	RMSE (€/MWh)	ES-like (€/MWh)	R^2
90	6	400	10	16.600	24.656	70.241	0.816
90	6	400	20	16.613	24.674	70.296	0.815
90	6	300	10	16.593	24.675	70.458	0.815

TABLE 4.5. BEST BASELINE RANDOM FOREST CONFIGURATIONS

[a]SW: Sliding Window
[b]LW: Lag Window

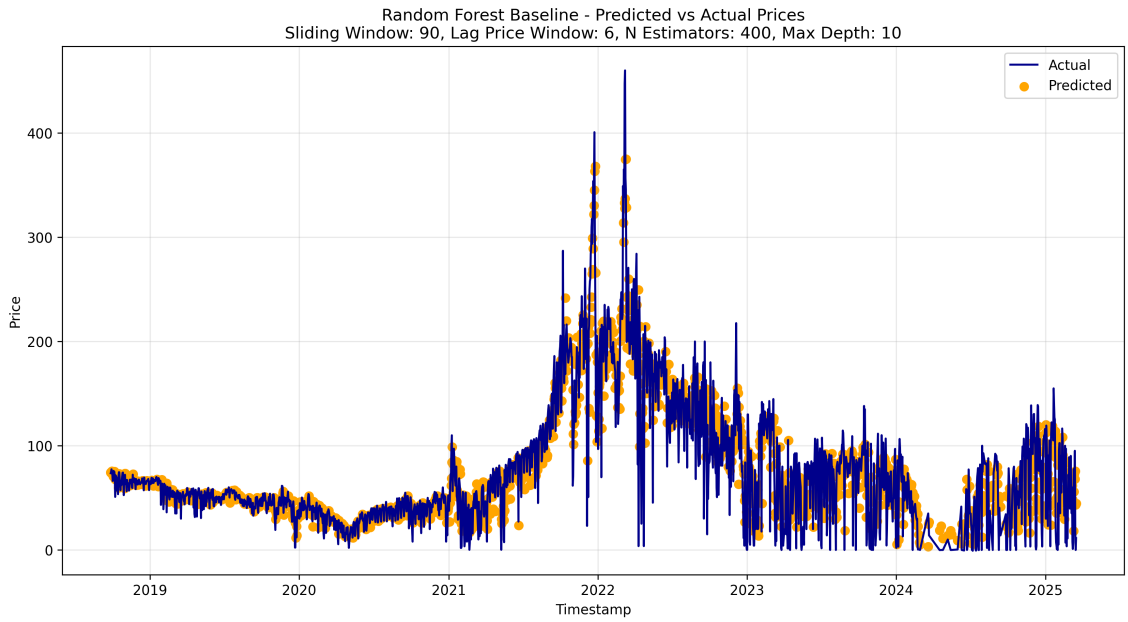

Fig. 4.7. Best Random Forest Baseline Model

The baseline Random Forest model shows an expected mixed performance. The model does

not successfully capture any of the major price spikes in 2022, this can be attributed to the previous explanation of the models averaged outputs, which are unlikely to catch peaks. Even though Random Forest can capture non-linear relationships that linear models miss, its nature appears to be overly conservative, rarely predicting extreme values and often "playing it safe" with middle-range predictions.

A couple of concerning behaviors are the following: throughout most periods, especially in the 2021 price escalation, where the predictions consistently fall below actual prices. There is also a noticeable lag in following price trends, observable in the 2021 rise, and in the 2024 price fall, where predictions appear to trail behind the actual values. Considering this best run used a larger than previous 90-day window, and a 6-lag price window, the model might not be fully exploiting the temporal information available. This suggests the model might need further tuning, such as experimenting with feature engineering to achieve more comparable performance to its linear counterparts. The following results display the best feature engineering runs, and the top performer plot:

SW[a]	**LW**[b]	**n_estimators**	**max_depth**	**MAE (€/MWh)**	**RMSE (€/MWh)**	**ES-like (€/MWh)**	**R^2**
90	1	400	10	15.804	23.572	68.080	0.834
90	1	400	20	15.814	23.589	68.123	0.834
90	1	400	30	15.814	23.589	68.123	0.834

TABLE 4.6. BEST FEATURE ENGINEERING RANDOM FOREST CONFIGURATIONS

[a]SW: Sliding Window
[b]LW: Lag Window

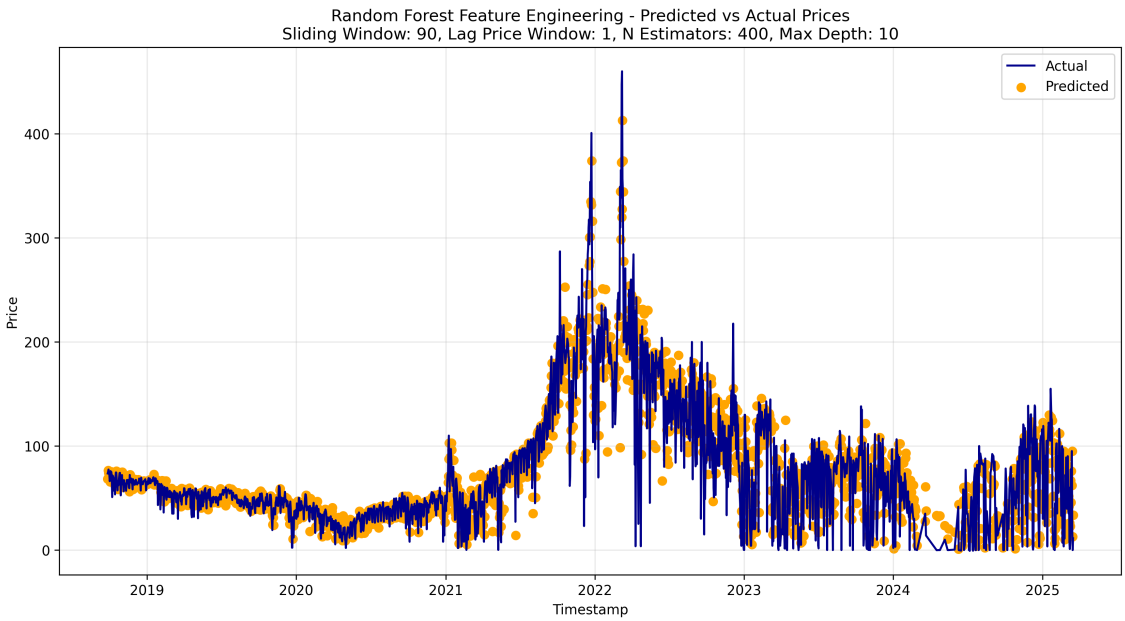

Fig. 4.8. Best Random Forest Feature Engineering Model

The Random Forest Feature Engineering model has slightly better performance metrics, but shows much more variability in predictions. This can be observed with the models leniency to make better peak estimations, improving on the higher predictions across the time series. But this comes at the cost of making a lot of underestimations, which simply did not happen in the baseline experiment. This exposes that feature engineering strategies need to be algorithm-specific, as success appears to be dependent on the model used, since the same approach that gave good results with the linear models failed to provide better training context for the Random Forest model.

Model Comparison

Concluding the main results overview, the following table presents the best performing run of each model based on the best *RMSE*, which is considered to be the most representative metric to use when comparing different models over the same dataset. In the case of the tie observed in linear regression, the *coefficient of determination* metric was used, since a higher R^2 indicates a better "predicted to actual" value fit.

Model	**Type**	**SW**[a]	**LW**[b]	**Hyperparameters**	**RMSE (€/MWh)**	**R^2**
Linear Regression	Baseline	30	1	-	22.798	0.842
Lasso Regression	Baseline	30	1	alpha = 0.01	22.798	0.842
Random Forest	Feature Engineering	90	1	n_estimators = 400, max_depth = 10	23.572	0.834

TABLE 4.7. BEST PERFORMING CONFIGURATION FOR EACH MODEL

[a]SW: Sliding Window
[b]LW: Lag Window

Reviewing the best results, we can conclude that the feature engineering efforts were not as effective as expected. The more complex data preparation resulted in a training process that was notably more computationally expensive, with exponentially increased run times across models, without yielding significant improvements. This can be highlighted with the best Lasso configurations, which all disregarded the regularization parameter, as the models behaved identically to the best Linear regression model. Concluding, the Random Forest feature engineering configurations performed better on average, but with too much variability in comparison to the baseline.

Whilst the feature engineering runs slightly edged the baseline models in the average error, they consistently worsened in overall accuracy. In the linear models, this was shown by a generally lower R^2 metric, and by a higher mean squared error, usually due to bigger outliers among the predictions. This is also backed up by our ES-like metric, where we can observe a near 10 point worsening of the 5% worst predictions. For the non-linear model, feature engineering improved metrics across the board, but introduced greater variability in the results. So it would not be correct to consider this a positive improvement, given that there are trade-off for each approach.

Note: The code for all of these experiments can be found here: (link github final code)

4.4. Discussion

In this section we present a comprehensive analysis of the data processing pipeline performance across all experiments. We compare all configurations, gathering insights about the impact of the different temporal window setups, the feature engineering process, and hyperparameter tuning. The evaluation primarily includes our calculated metric, MAE, MSE, R^2, and our Expectation Shortfall-like metric to capture the models' performance across all possible combinations. Once again, the following results are for just the 99th percentile in order to best represent the actual models performance, ignoring isolated model errors.

For clarity during the following figure descriptions, "longer" or "shorter" sliding windows, will refer to the number of days included in the training, and "wider" or "narrower" lag price windows, will refer to the number of feature columns that the training matrix included. For the error metrics (MAE, MSE, ES-like) a lower number is better, while for R^2 the closer to 1, the better.

Linear Regression

Being linear regression our first model, no hyperparameters were considered, simplifying the distribution representation of the performance metrics. As we can only tune two parameters, `sliding_window` and `lag_price_window`, the configurations can be plotted over a 2D heat map. The figure representing the the baseline model distribution is the following:

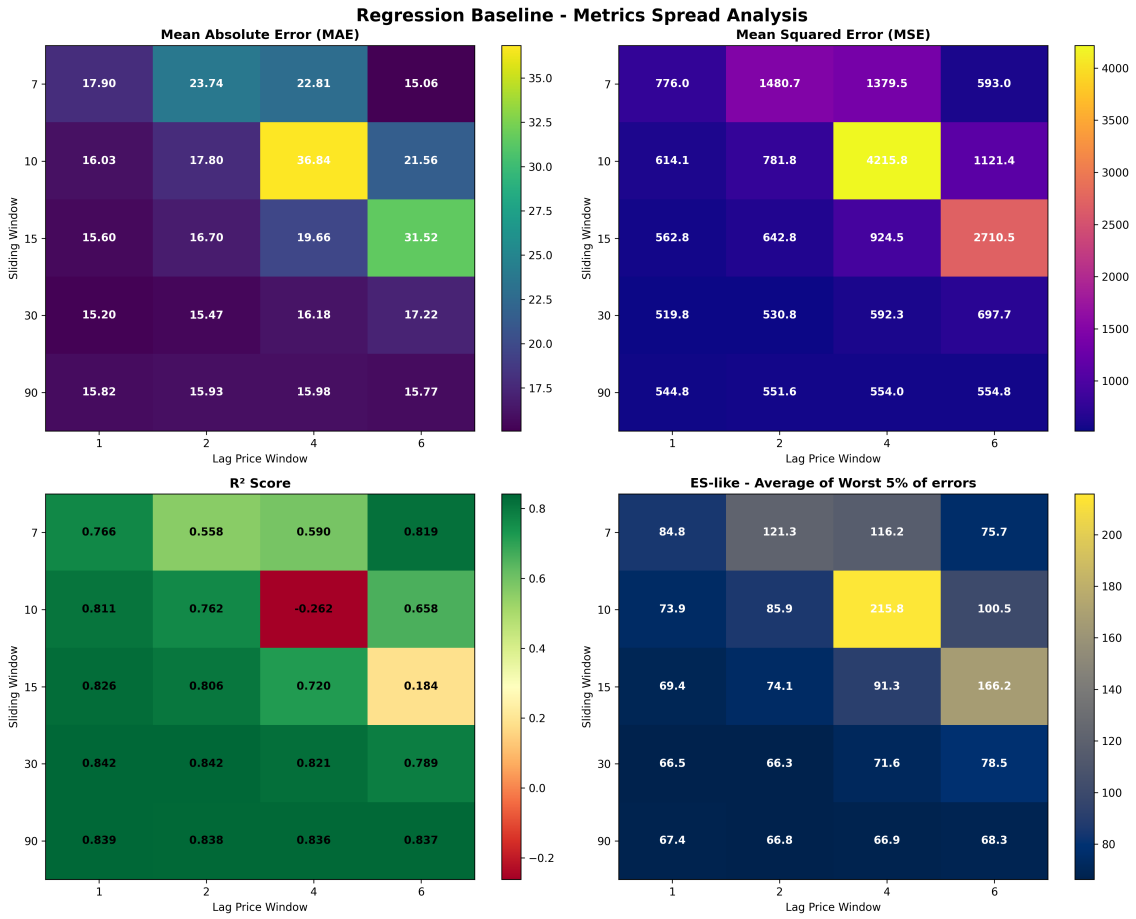

Fig. 4.9. Baseline Linear Regression Metrics Distribution

For the baseline models, we can observe the distribution of temporal windows is skewed towards the taller, but simpler feature matrices. The previously discussed top three results are located in the bottom left of the temporal window heatmaps. In general, we can observe that the longer sliding windows, with the least features are preferable, followed by longer sliding windows combined with more features. There is an exception to this, which is the 7-day sliding window with 6 lag days, which does also have fairly low error rates, and slightly lower accuracy than the best performers.

For the more complex training including the technical indicators we can calculate the following heatmaps:

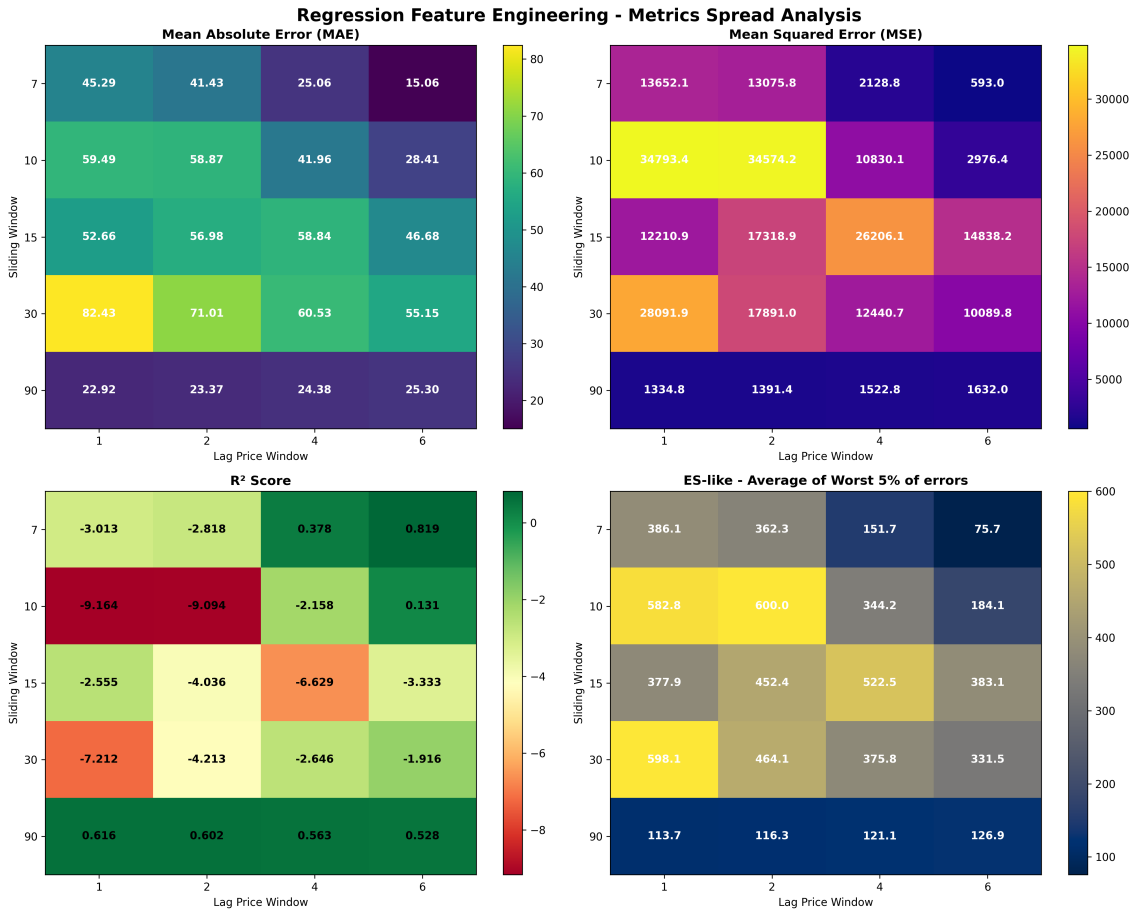

Fig. 4.10. Feature Engineering Linear Regression Metrics Distribution

A visual inspection clearly shows that the best temporal window combination is the 7-day by 6 lag price matrix, as discussed in the previous section. In general, taller and narrower matrices are no longer preferable, with the exception of the 90-day sliding windows, which are the next best performers, ordered from narrower to wider. This indicates that the technical indicators are managing to transfer the temporal characteristics over to the model, such as the trend, momentum or volatility. This can be inferred as the optimal configuration uses a shorter sliding window, meaning fewer actual prices are included, therefore the model must be basing its decisions on the temporal features, and the previous day context given by the widest lag price window.

Lasso Regression

To be able to understand the lasso result distribution, we must consider and tune its hyper-parameter *alpha*. The best score results for each temporal window combination are presented, displaying below the *alpha* parameter with which they were obtained with. An additional heat map will be included to exclusively display the most common *alpha* values across the different temporal window configurations. This will attempt to provide the extra dimensionality required to understand the Lasso regression results. Starting with the baseline results, the following figure displays the best results for each window, and the respective *alpha* with which they were obtained:

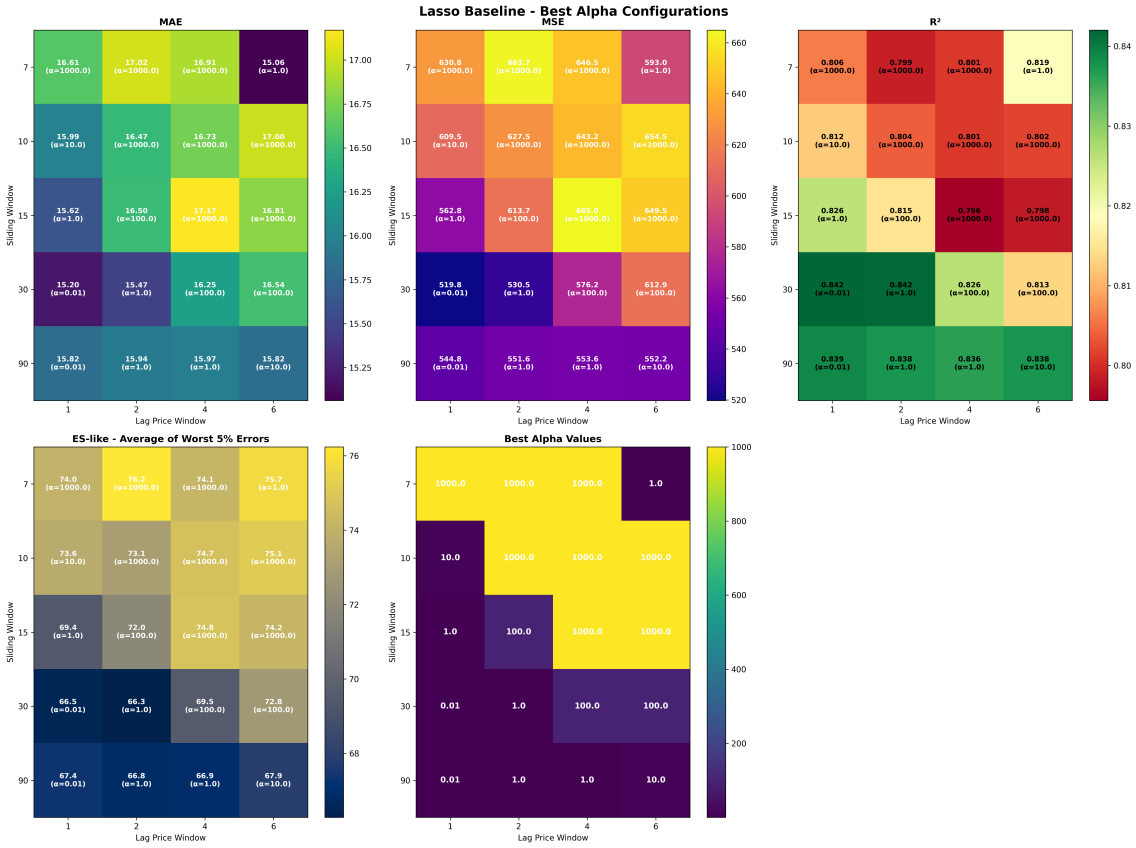

Fig. 4.11. Baseline Lasso Best Alpha Metrics Distribution

For the the analysis of the Lasso model, the hyperparameter value selection is probably the most relevant result. In the case of these baseline results, we can observe that the best *alpha* is the lowest value considered in the training batches. An *alpha* of 0.01 barely introduces any regularization, meaning instead of reducing the effect of certain features to zero, it considers them all as relevant, effectively working as Linear regression. The results are in line with the temporal window dimensionality, since the narrower the lag day window is, the less it can regularize. This is due to the fact that in the baseline runs, the feature matrix is as wide as the lag price window value, as it does not have any other features. These results make sense since the Lasso model cannot regularize properly when it has fewer features. Irrespective of this, we can observe that the best performing temporal window configuration is the exact same as with Linear regression, indicating that the Lasso model was unable to properly utilize lag day prices as features to aid in

the model training.

The following figure presents the results of the feature engineering training batches:

Fig. 4.12. Feature Engineering Lasso Best Alpha Metrics Distribution

In this last linear model analysis can observe that the best performance was obtained in the 7-day sliding window, with 6 lag day prices. In this occasion, as discussed in the previous section, the top performers used the same temporal window, but with very different *alpha* values. This best result can be observed with the lowest MAE, MSE and highest, most accurate, R^2 score, with the exception of the ES-like metric, which as towards the higher end of the values. As mentioned earlier, this indicates that the feature engineering selection was optimal, as the results were independent of the *alpha* value. Concluding with the linear models analysis, the regularization of the Lasso hyperparameter was not crucial in the performance of the model.

Random Forest

Our final analysis will review the Random Forest regressor. In this non-linear model we will have with one more parameter than with Lasso regression, adding more complexity to the results. The first parameter will analyze the effect of tuning the number of estimators, `n_estimators`, arguably the most important parameter, which determines the number of "trees" that will be built in the forest. We will also consider the effect of limiting the maximum depth, `max_depth`, of each decision tree. Tuning this helps to prevent individual trees from becoming too complex and overfitting to the training data, which contributes to the overall model's generalization ability.

The baseline training batch results are shown in the following figure, including two more graphs depicting the best parameter selection for each window:

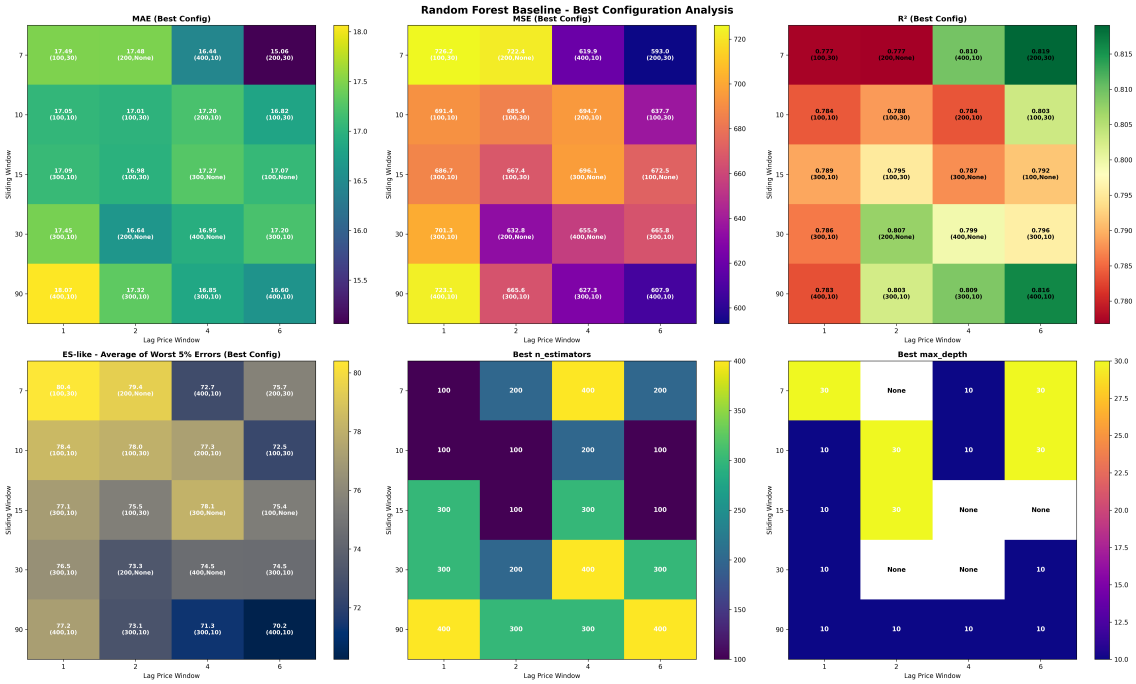

Fig. 4.13. Baseline Random Forest Metrics Distribution

In these results we can observe that the best result that was obtained was for the 90-day sliding window, with the 6-day lag price window. The non-linear nature of the model clearly benefited the most out of more features in this baseline run with the simple matrix. Once again, the top results are complex to display, as they employed the same temporal window, with different `n_estimators` and `max_depth` parameters.

Due to the increased complexity of the previous results, one more plot will be included. To aid in the understanding of the results, the following combined chart displays the frequency of each parameter among the best performing parameter combinations:

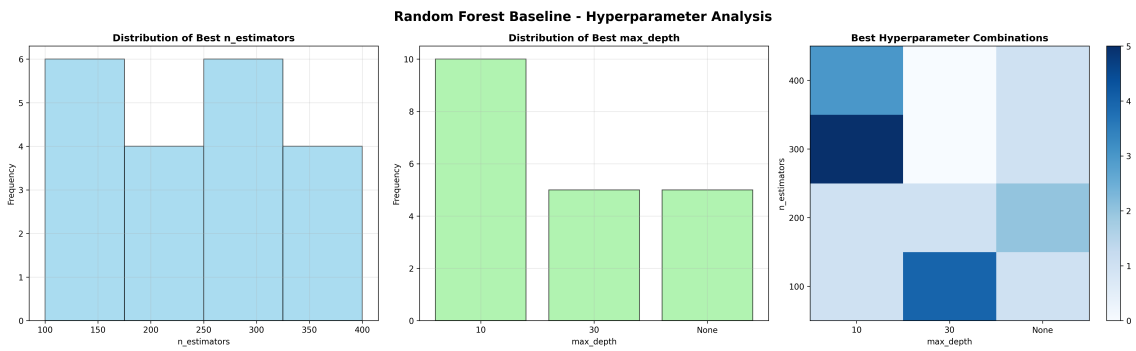

Fig. 4.14. Baseline Random Forest Hyperparameter Analysis

From these distributions we can observe that lower tree depths were typically preferred among the top performers, which were not as sensitive to the number of estimators shown in the mixed `n_estimators` results. The best performing models clearly employed parameter combinations from the top results, with 300 estimators actually being more prevalent among them, rather than the 400 estimators employed by the two best models. These are interesting findings that proved that for this sliding window approach, a higher number of estimators, with shallower tree depths were optimal for the best predictors on average.

For the final training batch, the feature engineering results are presented in the following figure:

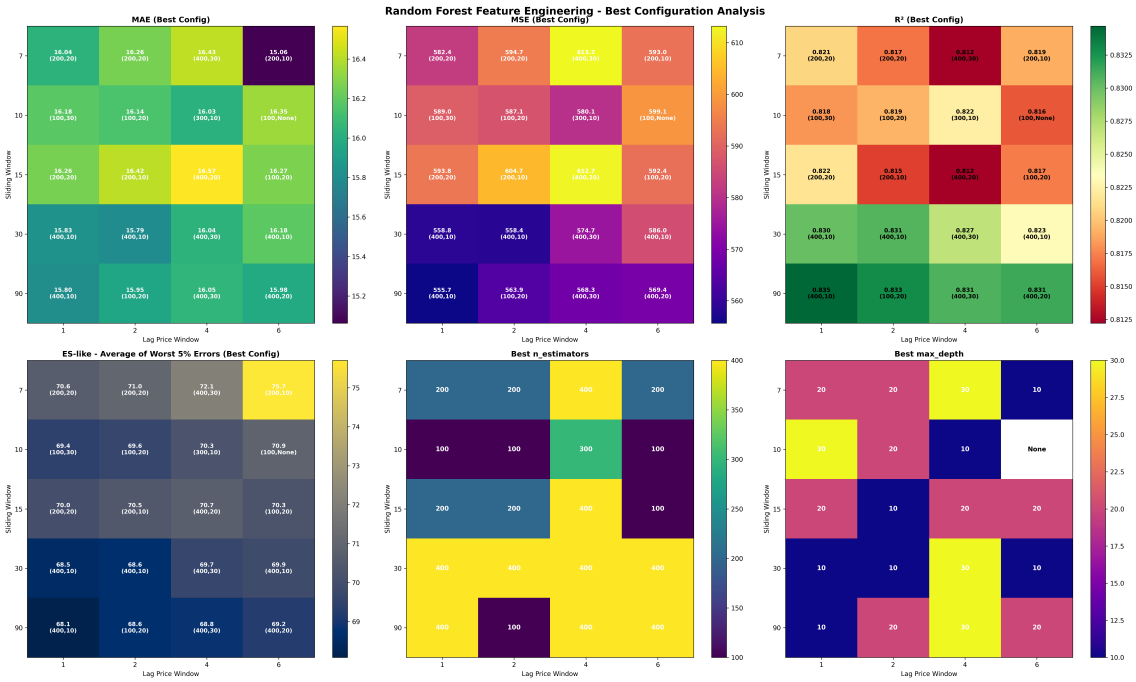

Fig. 4.15. Feature Engineering Random Forest Metrics Distribution

For the feature engineering analysis, we can observe that the longer but narrower temporal window configurations were preferred. This aligns well with the nature of the extended matrix, as it is formed with an extensive number of feature columns, from which the non-linear model can attempt to extract the complex temporal relations encoded in the technical indicators, depending less on the actual price time series. As in the previous analysis, the best results are once again using the same temporal windows, slightly complicating their visualization in the graphs. The 90-day sliding window, combined with just a 1-day lag price feature, is the top performer. These models used the maximum number of estimators considered, and were mostly independent of the tree depth, with the shallower one slightly outperforming the others.

For our final in depth discussion figure, here are the best, most frequent combinations of parameters for the Random Forest model:

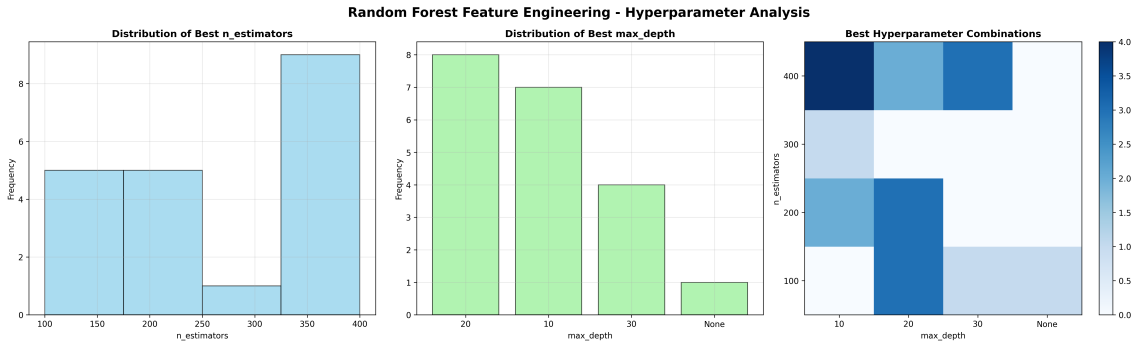

Fig. 4.16. Feature Engineering Random Forest Hyperparameter Analysis

These final distributions display show that the most prevalent number of estimators among the top performers was the highest 400 value. They also show that not setting a maximum tree depth was detrimental to the model, as most of them preferred at least a fixed depth. These findings prove that with an extensive selection of features, a higher number of estimators is needed to correctly extract all of the temporal information, and a fixed dept is needed to avoid overfitting, given the best models employed the longest 90-day sliding window considered in our experiments

The Effect of Feature Engineering

As a final remark for the discussion of the results, the use of feature engineering provided mixed results, as it was generally not as beneficial to the as expected. The linear models improved slightly on their MAE and MSE, but worsened their precision (R^2 score), along with an increase in their ES-like average of the 5% of worst errors. Only the non-linear Random Forest was able to improve across all metrics, but at the cost of increased variability among its predictions, highlighting that it would have probably benefited from a model-specific feature engineering process.

The code that formed the basis of this analysis can be found here: (link github final code)

5. REGULATORY FRAMEWORK

In any data-driven project understanding the regulatory landscape is essential. This chapter addresses the legal, ethical, and operational constraints that governed the use of data and software throughout the development of this project. From the data acquisition and pre-processing, to the software deployment and reproducibility, each component aims to comply with relevant national and international standards to ensure a responsible and legal development.

5.1. Data Availability

The dataset used in this project was obtained from the OMIE, a publicly traded company designated as the Iberian energy market operator. This organization publishes the energy market data as part of their legal obligations set by the Spanish CNMC for transparency, aligned with European regulations.

For the purpose of this thesis, which is strictly academic and investigative in nature, Spanish and European regulations allow for the use of publicly available institutional data without requiring additional licenses. This includes data shared under open access or public sector information frameworks, specifically for research, education, or non-commercial work.

However, it is important to note that if the project were to evolve into a commercial application, the legal requirements would change. In such a scenario, additional licensing agreements, permissions for commercial use, and compliance with intellectual property laws may be necessary.

While no personal or sensitive data is used in this project, future developments that incorporate customer or proprietary data would also need to comply with GDPR and ethical standards for responsible AI and data use, especially in markets as sensitive as energy trading.

5.2. Software & Licenses

This project was developed using a combination of open-source and proprietary software tools. In the context of regulatory and legal frameworks, the selection and usage of these tools are not only technical decisions, they also bear implications related to licensing compliance.

To ensure transparency and reproducibility, all core dependencies have been documented. Where applicable, licensing schemes have been reviewed to ensure compatibility with both academic and industrial research standards.

For clarity, the software tools have been categorized into two distinct parts: required software to reproduce the project, and optional tools to enhance the development workflow.

Required Software:

- Python: Primary programming language used throughout the project. Python is open source under the Python Software Foundation License [17].

- Pandas: Data manipulation library, distributed under a variety of permissive open-source licenses [18].
- NumPy: Numerical computing library, distributed under its own open-source license [19].
- Scikit-learn: Machine learning library, used for model training and evaluation. [20].
- ta (Technical Analysis) Library: Used for calculating financial indicators to aid in the time series analysis. It is available under the MIT license, a permissive open-source license [15].
- Matplotlib: Used for data visualization, distributed under a permissive open source license [21].
- macOS: Closed-source operating system on which development was performed. Its usage is subject to commercial a licensing agreement by Apple Inc. [22].

Optional Software:

- Git: Open-source version control system licensed under GPLv2, used for code tracking and collaboration [23].
- Git: Open-source distributed version control system, used for code tracking and collaboration [23].
- GitHub: Repository hosting platform for Git repositories. Though it hosts open-source projects, GitHub itself is a proprietary service governed by its own Terms of Service [24]
- Visual Studio Code: Open-source code editor under the MIT license, provided by Microsoft [25].
- Visual Studio Code Extension: Microsoft Data Wrangler: A proprietary extension that provides pandas.DataFrame visualization capabilities during development [26].
- Python Virtual Environment (venv): Used to make the project environment reproducible. It is best practice to create new Python virtual environments for each new project, specially in a production environment where certain versions are required in order for all components to work as intended [11]. An alternative method of dealing with this would be using Conda, and creating with it a new Conda environment [12].
- Overleaf: Cloud-based LaTeX editor used to write this document. Overleaf is a closed-source platform with cloud-hosting considerations that may have implications under data protection regulations such as the GDPR [27]
- LaTeX: Open-source typesetting language used to produce the final project document. Licensed under its own LaTeX Project Public License [28].
- OpenAI ChatGPT, Google Gemini & Anthropic Claude: Closed source large language models used to aid in general code troubleshooting. Licensed under their own respective terms of use.

6. SOCIO-ECONOMIC ENVIRONMENT

Energy markets play a central role in modern economies, directly affecting industrial output, consumer costs, and overall economic stability. Electricity is a resource that cannot be easily stored at scale. Its demand fluctuates continuously, and supply must adapt almost instantaneously. As a result, accurate energy price prediction is not just a technical challenge, but a key enabler for operational efficiency, market stability, and policy decision-making.

This project operates within this socio-economic context. Understanding the broader environment highlights its relevance and the potential future applications of the work. The forecasting models developed here could contribute to better-informed decision making across multiple stakeholders, from grid operators and energy producers to regulators, businesses, and end consumers.

6.1. Impact

Accurate energy price forecasting carries significant socio-economic implications. For instance, improved price prediction allows both producers and consumers to optimize their operations. Producers can better plan generation schedules, while industrial consumers may adjust consumption to minimize operating costs. Many sectors, such as the metal, cement, glass and chemical industries, are highly sensitive to energy price variations. Even small deviations in price forecasts can lead to significant financial variations that affect margins and ultimately the market price of goods such as raw building materials or gasoline.

Another example sector that consumes vast amounts of electricity is the telecommunications industry, along with its data centers and IT services. In today's modern society, we cannot afford to ever shut down crucial communications infrastructure, or data center servers, which aim for the highest possible availability. The electric mobility and logistics sector is another key example of an industry that is not tolerant to shutdowns and failures, since they depend on easy and relatively affordable access to electricity. For these various sensitive industries, forecasting models capable of anticipating these fluctuations can provide a buffer for strategic planning, risk management, and preventive measures.

In this project, machine learning techniques were applied to develop predictive models capable of capturing the complex patterns of historical energy price data. By doing so, the work aims to contribute not only to academic research, but also to the development of real, "production" tools that may offer real-world benefits within the larger socio-economic ecosystem of energy markets.

6.2. Project Plan

The following Gantt diagram outlines the approximate time frames of each project phase. This estimation has been done according to the actual design and development process. Many of the phases of the project overlap each other due to the technical complexity of translating the theoretical data processing pipeline to the python code that executed the different steps.

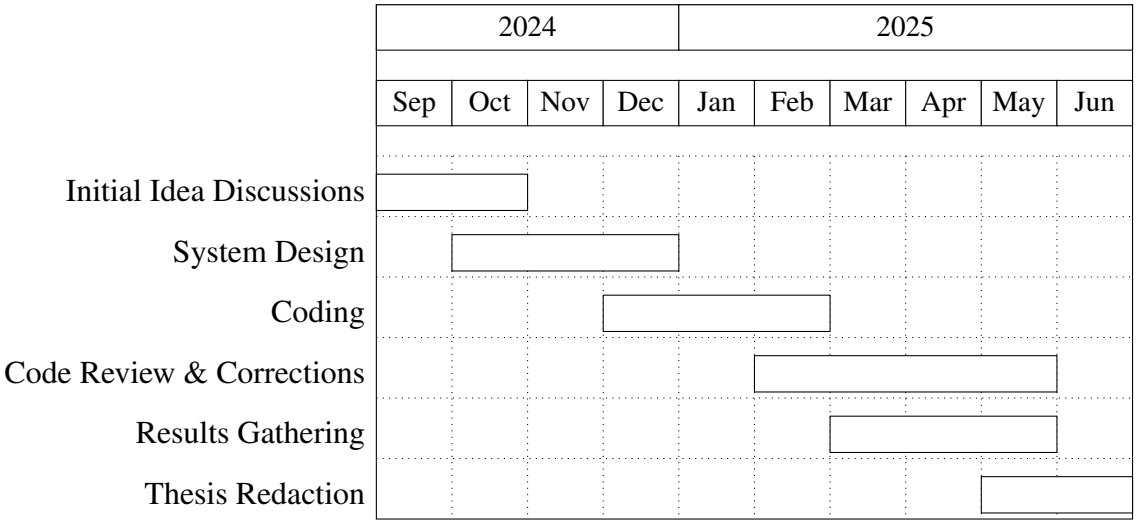

Fig. 6.1. Gantt Diagram of the Project Lifecycle

6.3. Budget

This section will consist of a complete price breakdown of the research, consisting of material costs such as the equipment utilized, and the human capital that composed the research team. It is notable to highlight that as previously mentioned, no paid license of any sort was obtained for the realization of this project. All of the programs, tools or code, were either *Open Source* or free to use software under exclusive *non-commercial use* licenses.

The most basic requirement for the physical needs of this project is a computer with an desktop operating system. Since the project was written in Python [17], it would be reasonable to consider it platform agnostic. Any computer capable of running any recent operating system, configured with a recent version of Python would be able to run this project. This includes any of the three most popular desktop operating systems, Windows, macOS and any Linux distribution.

The project was mainly tested using Python 3.13 [10] which does require a recent operating system such as macOS 13 or Windows 10. But as for the project itself, there is no platform, processor architecture, or processing power requirements. In my case, I mainly developed the code in my personal laptop, a 2021 MacBook Pro configured with the ARM M1 Pro chip and 16GB of RAM, running the latest available macOS version at this time, macOS 15 [22]. This laptop was obtained from a second-hand store circa June 2024, mainly for personal use, but also helping keep the theoretical project costs down.

The following table outlines the specific equipment costs incurred for the development of this research project:

Equipment	Price (€)	Amortization per year (€)	Useful Life (years)	Usage Time (months)	Cost (€)
MacBook Pro M1 Pro	1.250	312.5	4	9.5	247.4

TABLE 6.1. EQUIPMENT AMORTIZATION

Regarding the human capital that was required for the successful development of this project, it will consist exclusively on two people. The director of my bachelor's thesis, professor Emilio Parrado Hernández, PhD, as the expert Machine Learning consultant, and myself as a junior software engineer.

The number of engineering hours was calculated by assuming the research was worked on a daily, with a 4h part time schedule, for an estimated 27 weeks, taking into account holidays and other extraordinary operational circumstances. For the consulting services, each project meeting will account for one billable hour.

The following table displays the estimated hourly rates of each person, applied to the approximate number of hours dedicated to the project:

Name	Role	Price (€/hour)	Hours	Cost (€)
Emilio Parrado Hernández	Expert ML Consultant (PhD)	200	22	4.400
Rodrigo De Lama Fernández	Junior Engineer (Pre-Graduate)	15	540	8.100

TABLE 6.2. HUMAN COSTS

For our final budgeting considerations, other miscellaneous expenses, such as transportation costs to the university for on site meetings with my bachelor's thesis director.

Summing up all costs, the following table estimates what this project would have cost to investigate:

Concept	Cost (€)
Direct/Material Costs	247.4
Engineering Costs	8.100
University Carlos III Costs - Expert ML Consultant	4.400
Total	**12.747,4**

TABLE 6.3. FINAL COST BREAKDOWN

7. CONCLUSIONS

In this research project we have analyzed in depth an innovative methodology for predictive modeling of energy prices with Machine Learning algorithms, enhanced by the usage of technical analysis indicators in feature engineering, in order to enhance prediction precision. This final chapter will review the general conclusions attained throughout the development of this project's Machine Learning based solution for energy price predictions. It will also discuss potential development paths for future work pertaining the system designed in this research project.

7.1. Revisiting the Objectives of the Research

After finishing the whole project, read it, and reintroduce the objectives to the reader. Remind them of the completion of them

Predicting the prices in an accurate manner using Machine Learning models using feature engineering with technical analysis indicators.

It did work - kind of:

Slight improvements in the Linear models, but with the trade-off of worse precision (R^2 values).

In Random Forest it worked better, but not amazing, specially for the compute required to process that model

7.2. Future Work

There are several promising directions to improve and extend this research project. Firstly, future efforts should focus on reducing the overall absolute error of the model, while also addressing the isolated error spikes that occasionally lead to significant deviations in prediction accuracy. This could be done by making adapting the temporal windows along the time series, for example a monthly or seasonal review and adjustment. Achieving smoother and more stable forecasts would result in better percentile accuracy across the entire distribution of predicted prices.

Another potential avenue is the development of a hybrid model architecture that combines the strengths of different algorithms. For example, integrating the consistency and robustness of Random Forests with the sensitivity of linear models would potentially enable capturing sharp price variations and peaks, creating a more balanced and accurate predictor.

Finally, with further validation and performance tuning, this project could evolve into a production ready system, potentially serving as the foundation for a commercial application capable of providing real-time electricity price forecasts to companies, traders, and consumers seeking to optimize their energy costs.

BIBLIOGRAPHY

[1] Jefatura del Estado, *Circular 3/2019, de 20 de noviembre, de la comisión nacional de los mercados y la competencia, por la que se establecen las metodologías que regulan el funcionamiento del mercado mayorista de electricidad y la gestión de la operación del sistema.* https://www.boe.es/diario_boe/txt.php?id=BOE-A-2019-17287, Dec. 2019.

[2] Government of Portugal and Government of Spain, *International agreement between the portuguese republic and the kingdom of spain for the creation of an iberian electricity market (mibel)*, https://www.omip.pt/system/files/2020-01/santiago_international_agreement_01.oct.2004_eng_01_0.pdf, Specifically, Article 4: Creation of an Iberian Market Operator, Oct. 2004.

[3] OMIE, *Funcionamiento del mercado diario*, https://www.omie.es/sites/default/files/inline-files/mercado_diario.pdf.

[4] The scikit-learn Development Team, *Scikit-learn: Linear regression*, https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LinearRegression.html.

[5] The scikit-learn Development Team, *Scikit-learn: Lasso regression*, https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.Lasso.html.

[6] The scikit-learn Development Team, *Scikit-learn: Random forest regressor*, https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestRegressor.html.

[7] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006. [Online]. Available: https://www.microsoft.com/en-us/research/wp-content/uploads/2006/01/Bishop-Pattern-Recognition-and-Machine-Learning-2006.pdf.

[8] S. Donadio and S. Ghosh, *Learn Algorithmic Trading: Build and deploy algorithmic trading systems and strategies using Python and advanced data analysis*. Packt Publishing, 2019.

[9] F. J. Nogales, J. Contreras, A. J. Conejo, and R. Espínola, "Forecasting next-day electricity prices by time series models," *IEEE Transactions on Power Systems*, vol. 17, no. 2, May 2002. [Online]. Available: https://halweb.uc3m.es/fjnm/esp/papers/TransferPrices.pdf.

[10] Python Software Foundation, *Python 3.13*, https://www.python.org/downloads/release/python-3130/.

[11] Python Software Foundation, *Venv — creation of virtual environments*, https://docs.python.org/3/library/venv.html.

[12] Anaconda Inc., *Miniconda*, https://www.anaconda.com/docs/getting-started/miniconda/main.

[13] OMIE, *Precios horarios del mercado diario en españa*, https://www.omie.es/es/file-access-list?parents=/Mercado%20Diario/1.%20Precios&dir=Precios%20horarios%20del%20mercado%20diario%20en%20EspaÃśa&realdir=marginalpdbc, Jun. 2025.

[14] The Pandas Development Team, *Pandas dataframe*, https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.html.

[15] bukosabino, *Technical analysis library in python (ta)*, https://github.com/bukosabino/ta, 2023.

[16] OMIE, *Modelo de ficheros para la distribución pública de información del mercado de electricidad 1.35*, https://www.omie.es/sites/default/files/2024-06/Formato_ficheros_inf_pub_135_0.pdf, Jun. 2024.

[17] Python Software Foundation, *Python programming language*, https://www.python.org/about/.

[18] The Pandas Development Team, *Pandas: Python data analysis library*, https://pandas.pydata.org.

[19] The NumPy Development Team, *Numpy: Array programming for scientific computing*, https://numpy.org.

[20] The scikit-learn Development Team, *Scikit-learn: Machine learning in python*, https://scikit-learn.org/stable/.

[21] Matplotlib Development Team, *Matplotlib: A comprehensive library for creating static, animated, and interactive visualizations in Python*, https://matplotlib.org/.

[22] Apple Inc., *Macos sequoia*, https://www.apple.com/macos/macos-sequoia/, 2024.

[23] Git, *Git: A distributed version control system*, https://git-scm.com.

[24] GitHub, *Github: Repository hosting platform*, https://github.com/about.

[25] Microsoft Corporation, *Visual studio code*, https://code.visualstudio.com/docs/editor/whyvscode, Open source code editor.

[26] Microsoft, *Data Wrangler: A code-centric data viewing and cleaning tool for Visual Studio Code*, https://marketplace.visualstudio.com/items?itemName=ms-toolsai.datawrangler.

[27] Overleaf, *Overleaf: Online LaTeX editor*, https://www.overleaf.com/about.

[28] LaTeX Project, *Latex: A document preparation system*, https://www.latex-project.org.

APPENDIX A: PROJECT ENVIRONMENT CONFIGURATION

Create the virtual environment:

```
python -m venv .venv
```

Activating the new virtual environment:

```
in macOS
source .venv/bin/activate

in Windows
.venv\Scripts\Activate.ps1
```

To reproduce the exact environment used in the project, a *requirements.txt* file must be created with the following contents:

```
ipykernel==6.29.5
matplotlib==3.10.1
numpy==2.2.3
pandas==2.2.3
requests==2.32.3
scikit-learn==1.6.1
ta==0.11.0
```

To install the dependencies run:

```
pip install -r requirements.txt
```

APPENDIX B: AUTOMATED DOWNLOAD OF OMIE RAW DATA

```
import requests
from datetime import datetime, timedelta

# Usage instructions:
    # Set file name to save the filenames to download
    # Set start and end dates

file_path = 'to_download.txt'
start_date = datetime(2023, 1, 1)
end_date = datetime(2025, 3, 18)

# Compose filenames to download
def compose_filenames():
    start_date = start_date
    end_date = end_date

    # Generate the entries for the number of days comprehended
    to_generate = (end_date - start_date).days + 1
    entries = []
    for i in range(0, to_generate):  # x days from start to end
        date_entry = start_date + timedelta(days=i)
        entries.append(f'marginalpdbc_{date_entry.strftime("%Y%m%d")
            }.1')

    # Save to a .txt file
    with open(file_path, 'w') as file:
        for entry in entries:
            file.write(entry + '\n')

# Example URI to build
# https://www.omie.es/es/file-download?parents%5B0%5D=marginalpdbc&
    filename=marginalpdbc_20230102.1
def read_filenames_and_compose_urls(file_path):
    base_url = "https://www.omie.es/es/file-download?parents%5B0%5D=
        marginalpdbc&filename="

    # Read the filenames from the .txt file
    with open(file_path, 'r') as file:
        filenames = [line.strip() for line in file if line.strip()]

    # Compose the URLs
    urls = [base_url + filename for filename in filenames]

    return urls

```

```python
def download_files(urls):
    for url in urls:
        # Extract the filename from the URL
        filename = url.split('=')[-1]

        # Send a GET request to download the file
        response = requests.get(url)

        # Check if the request was successful and write to file
        if response.status_code == 200:
            # Write the content to a file with the extracted filename
            with open(filename, 'wb') as file:
                file.write(response.content)
            print(f"Downloaded {filename}")
        else:
            print(f"Failed to download {filename}")

# Execution
compose_filenames()
urls = read_filenames_and_compose_urls(file_path)
download_files(urls)
```

APPENDIX C: DATA INGESTION AND CLEANUP

```
import pandas as pd
import os

# Base path to the folder containing the year by year data folders
base_path = '../TFG/data/'

# Initialize an empty list to store DataFrames
all_data = []

# Iterate through each year folder and process all files
for root, dirs, files in os.walk(base_path):
    for file in files:
        # Check if the file starts with 'marginalpdbc_' (ignoring the rest, including the extension)
        if file.startswith('marginalpdbc_'):

            # Read the file
            file_path = os.path.join(root, file)
            with open(file_path, 'r', encoding='utf-8') as f:
                lines = f.readlines()

            # Remove the first line (MARGINALPDBC;) and the last line (*)
            data_lines = lines[1:-1]

            # Convert the list of semicolon-separated strings into a DataFrame
            daily_data = pd.DataFrame([line.strip().split(';') for line in data_lines])

            # Remove trailing ';' from the last column
            if daily_data.shape[1] > 6:
                # Drop empty columns
                daily_data = daily_data.iloc[:, :6]

            # Assign column names
            daily_data.columns = ['Year', 'Month', 'Day', 'Hour', 'MarginalPT', 'MarginalES']

            # Remove rows with invalid data (in case accidental empty rows, or non-numeric values, get included)
            daily_data = daily_data[daily_data['Year'].str.isdigit()]  # Only keep rows where 'Year' is a number

            # Convert the columns to the appropriate data types
```

```
            daily_data = daily_data.astype({
                'Year': int, 'Month': int, 'Day': int, 'Hour': int, 'MarginalPT': float, 'MarginalES': float
                })

            # Append daily data to the list
            all_data.append(daily_data)

# Concatenate all daily DataFrames into one big DataFrame
full_data = pd.concat(all_data, ignore_index=True)

# Create a datetime column from Year, Month, Day, and Hour
# Adjust the 'Hour' column to deal with the potential 25th hour issue in case of timezone adjustment
full_data['Hour'] = full_data['Hour'].apply(lambda x: 0 if x == 25 else x)
full_data['Datetime'] = pd.to_datetime(full_data[['Year', 'Month', 'Day', 'Hour']], errors='coerce')

# Set the 'Datetime' column as the index
full_data.set_index('Datetime', inplace=True)

# Drop the now unnecessary time columns and 'MarginalPT', as we are focusing on the Spanish market
full_data.drop(['Year', 'Month', 'Day', 'Hour', 'MarginalPT'], axis=1, inplace=True)

# Save the full dataset to a CSV file
full_data.to_csv('../../data/raw_data.csv')

# Post-ingest database cleanup:
# Load and sort the data by the 'Datetime', and save the sorted data to a new file
df = pd.read_csv('../../data/raw_data.csv')
df = df.sort_values(by='Datetime')
df.to_csv('../../data/processed_data.csv', index=False)

# Read the CSV file
df = pd.read_csv('../../data/processed_data.csv')
df['Datetime'] = pd.to_datetime(df['Datetime']) # Parse 'Datetime' column to datetime objects

# Generate the complete range of hourly timestamps
full_range = pd.date_range(start=df['Datetime'].min(), end=df['Datetime'].max(), freq='h')

# Find any missing timestamp
missing_timestamps = full_range.difference(df['Datetime'])

# If successful, print a message
if missing_timestamps.empty:
```

```
    print("Data ingestion completed successfully.")

# Else, print an error message
else:
    print("Error: Missing timestamps found. Check the data.")
    print("Missing timestamps:\n missing_timestamps")

    print("Data ingestion completed with errors.")
```

APPENDIX D: DATASET REDUCTION TO THE 14TH HOUR

```
import pandas as pd

# Load the original dataset
data_path = '../../data/processed_data.csv'
df = pd.read_csv(data_path, parse_dates=['Datetime'])

# Define the hour to predict and filter down to the 14:00H data
    point
hour_to_predict = 14
df_hour = df[df['Datetime'].dt.hour == hour_to_predict].copy()

# Save the final database
df_hour.to_csv("../../data/hour_14_metrics.csv", index=False)
```

APPENDIX E: TECHNICAL ANALYSIS INDICATORS CALCULATIONS

```
import pandas as pd
import ta  # bukosabino's Technical Analysis Library
import numpy as np

# Load the data
data_path = 'data/processed_data.csv'
df = pd.read_csv(data_path, parse_dates=['Datetime'])

# Reduce dataset to only to the 14th hour
hour_to_predict = 14
df = df[df['Datetime'].dt.hour == hour_to_predict].copy()

# The "window" parameter is measured in days

# Trend-following - SMA and EMA
# Simple Moving Average
df[f'SMA_{3}'] = ta.trend.SMAIndicator(close=df['MarginalES'],
    window=3).sma_indicator()
df[f'SMA_{5}'] = ta.trend.SMAIndicator(close=df['MarginalES'],
    window=5).sma_indicator()
df[f'SMA_{7}'] = ta.trend.SMAIndicator(close=df['MarginalES'],
    window=7).sma_indicator() # past week trend
df[f'SMA_{14}'] = ta.trend.SMAIndicator(close=df['MarginalES'],
    window=14).sma_indicator()
df[f'SMA_{30}'] = ta.trend.SMAIndicator(close=df['MarginalES'],
    window=30).sma_indicator() # past month trend
df[f'SMA_{90}'] = ta.trend.SMAIndicator(close=df['MarginalES'],
    window=90).sma_indicator()
df[f'SMA_{180}'] = ta.trend.SMAIndicator(close=df['MarginalES'],
    window=180).sma_indicator() # past half year trend

# Exponential Moving Average
df[f'EMA_{3}'] = ta.trend.EMAIndicator(close=df['MarginalES'],
    window=3).ema_indicator()
df[f'EMA_{5}'] = ta.trend.EMAIndicator(close=df['MarginalES'],
    window=5).ema_indicator()
df[f'EMA_{7}'] = ta.trend.EMAIndicator(close=df['MarginalES'],
    window=7).ema_indicator()
df[f'EMA_{14}'] = ta.trend.EMAIndicator(close=df['MarginalES'],
    window=14).ema_indicator()
df[f'EMA_{30}'] = ta.trend.EMAIndicator(close=df['MarginalES'],
    window=30).ema_indicator()

# Momentum - Rate of Change (ROC) and RSI
```

```python
# Rate of Change
df[f'ROC_{3}'] = ta.momentum.ROCIndicator(close=df['MarginalES'], window=3).roc()
df[f'ROC_{5}'] = ta.momentum.ROCIndicator(close=df['MarginalES'], window=5).roc()
df[f'ROC_{7}'] = ta.momentum.ROCIndicator(close=df['MarginalES'], window=7).roc()
df[f'ROC_{14}'] = ta.momentum.ROCIndicator(close=df['MarginalES'], window=14).roc()
df[f'ROC_{30}'] = ta.momentum.ROCIndicator(close=df['MarginalES'], window=30).roc()

# Relative Strength Index
df[f'RSI_{5}'] = ta.momentum.RSIIndicator(close=df['MarginalES'], window=5).rsi() # rapid momentum changes
df[f'RSI_{7}'] = ta.momentum.RSIIndicator(close=df['MarginalES'], window=7).rsi()
df[f'RSI_{14}'] = ta.momentum.RSIIndicator(close=df['MarginalES'], window=14).rsi() # biweekly momentum
df[f'RSI_{30}'] = ta.momentum.RSIIndicator(close=df['MarginalES'], window=30).rsi()

# Volatility - Bollinger Bands Width and Standard Deviation
# BB Width (Bollinger Bands Width)
df['BB_Width_7'] = ta.volatility.BollingerBands(close=df['MarginalES'], window=7, window_dev=2).bollinger_wband()
df['BB_Width_14'] = ta.volatility.BollingerBands(close=df['MarginalES'], window=14, window_dev=2).bollinger_wband()

# Standard Deviation (STD)
df['STD_7'] = df['MarginalES'].rolling(window=7).std()
df['STD_14'] = df['MarginalES'].rolling(window=14).std()
df['STD_30'] = df['MarginalES'].rolling(window=30).std()
df['STD_90'] = df['MarginalES'].rolling(window=90).std()

# Additional contextual features
df['month'] = df['Datetime'].dt.month
df['day_of_week'] = df['Datetime'].dt.dayofweek  # Monday=0
df['is_weekend'] = df['day_of_week'].isin([5, 6]).astype(int)

# Drop missing values generated by rolling calculations
df.dropna(inplace=True)

# Check for NaN or infinity values (specially in ROC)
print("NaN values per column:\n", df.isna().sum())
print("Infinity values per column:\n", df.isin([np.inf, -np.inf]).sum())

# Drop NaN or infinity values and interpolate
df = df.replace([np.inf, -np.inf], np.nan).dropna()
df = df.interpolate()
```

```python

# Save the dataset with metrics
df.to_csv(f'data/ta_metrics/final_price_ta_metrics.csv', index=False
    )
```

APPENDIX F: SIMPLE FEATURE MATRIX FUNCTION AND EXAMPLE RUN

Function:

```
import pandas as pd
import numpy as np

def create_feature_matrix(data, lag_price_window):
    """
    Creates a feature matrix where each row is a sliding window of prices,
    and a corresponding target vector containing the next price value.

    Parameters:
    ----------
    - data: DataFrame with 'Datetime' and 'MarginalES' columns
    - lag_price_window: Size of the previous prices sliding window

    Returns:
    -------
    - X: DataFrame with sliding windows as rows
    - y: Series with target values (next price after each window)
    """
    # Extract the MarginalES column
    if 'MarginalES' in data.columns:
        prices = data['MarginalES'].values
    else:
        # Assume it's the second column (index 1)
        prices = data.iloc[:, 1].values

    # Create empty matrices
    X = np.zeros((len(prices) - lag_price_window, lag_price_window))
    y = np.zeros(len(prices) - lag_price_window)

    # Fill the matrices with the sliding windows and targets
    for i in range(len(prices) - lag_price_window):
        X[i, :] = prices[i:i+lag_price_window]  # Window of previous prices
        y[i] = prices[i+lag_price_window]       # Next price after window

    # Convert to DataFrame/Series for easier use in training
    return pd.DataFrame(X), pd.Series(y)
```

Example run:

```python
import pandas as pd
from utils.matrix_builder import create_feature_matrix

# read data
csv_hour_file = '../data/ta_metrics/final_price_ta_metrics.csv'
simple_df = pd.read_csv(csv_hour_file, parse_dates=['Datetime'])
simple_df = simple_df[['Datetime', 'MarginalES']]

# Sliding window of 6 days
train_start_date = '2019-01-01'
train_end_date = '2019-01-07'

simple_subset = simple_df[(simple_df['Datetime'] >= train_start_date
    ) & (simple_df['Datetime'] <= train_end_date)]

# Number of previous days as columns (features)
lag_price_window = 3

X_simple, y_simple = create_feature_matrix(simple_subset,
    lag_price_window)
```

APPENDIX G: EXTENDED FEATURE MATRIX FUNCTION AND EXAMPLE RUN

Function:

```
def create_ta_feature_matrix(dataframe, lag_price_window):
    """
    Creates a sliding window dataset for time series forecasting where each row contains:
    1. A window of historical prices (right-aligned)
    2. Feature values from the most recent point in the window
    3. Target value (next price after the window)

    Parameters:
    ----------
    dataframe : pandas.DataFrame
        DataFrame containing at minimum 'Datetime' and 'MarginalES' columns,
        plus any additional feature columns
    lag_price_window : int
        Number of data points to include in each window of previous prices

    Returns:
    -------
    X : pandas.DataFrame
        Features DataFrame with:
        - Historical prices labeled as 'price_t-n' through 'price_t-1'
        - All additional features from the original dataframe
    y : pandas.Series
        Target values (price at time t)
    """

    # Input validation
    if lag_price_window < 1:
        raise ValueError("Feature window size must be at least 1")
    if len(dataframe) <= lag_price_window:
        raise ValueError(f"DataFrame must have more rows ({len(dataframe)}) than lag_price_window ({lag_price_window})")
    if 'Datetime' not in dataframe.columns or 'MarginalES' not in dataframe.columns:
        raise ValueError("DataFrame must contain 'Datetime' and 'MarginalES' columns")

    X, y = [], []
```

```python

    # Extract price data and features
    df_prices = dataframe[['Datetime', 'MarginalES']]
    df_features = dataframe.iloc[:, 2:] # Exclude 'Datetime' and 'MarginalES'
    feature_names = df_features.columns.tolist()

    # Create samples from the data
    for i in range(lag_price_window, len(df_prices)):
        # Extract the window for prices as features (right-aligned)
        window = df_prices.iloc[i-lag_price_window:i, 1].values.flatten()

        # Extract corresponding feature row (from the most recent point in the window)
        feature_row = df_features.iloc[i-1].values.flatten()

        # Concatenate window prices with feature row
        X.append(np.concatenate((window, feature_row)))
        y.append(df_prices.iloc[i, 1])  # Predict current price

    # Create column names for the prices feature window
    price_columns = [f'price_t-{lag_price_window-i}' for i in range(lag_price_window)]

    # Return DataFrame and Series with proper column names
    X_df = pd.DataFrame(X, columns=price_columns + feature_names)
    y_series = pd.Series(y, name='price_t')
    return X_df, y_series
```

Example run:

```python
import pandas as pd
from utils.matrix_builder import create_feature_matrix

# read data
csv_hour_file = '../data/ta_metrics/final_price_ta_metrics.csv'
features_df = pd.read_csv(csv_hour_file, parse_dates=['Datetime'])

# Sliding window of 6 days
train_start_date = '2019-01-01'
train_end_date = '2019-01-07'

feature_subset = features_df[(features_df['Datetime'] >= train_start_date) & (features_df['Datetime'] <= train_end_date)]

# Number of previous days as columns (features)
lag_price_window = 3

X_extended, y_extended = create_expanded_feature_matrix(
```

```
    feature_subset, lag_price_window)
```

APPENDIX H: EXAMPLE OF THE BASIC TRAINING PROCESS

```python
import pandas as pd
import numpy as np
from utils.matrix_builder import create_feature_matrix

# Import your preferred Machine Learning model here
from model_library import preferred_model

# Load Database
csv_hour_file = '../data/ta_metrics/final_price_ta_metrics.csv'
df = pd.read_csv(csv_hour_file, parse_dates=['Datetime'])

def generic_training(df, sliding_window, lag_price_window, DEBUG):
    """
    Train a machine learning model using a sliding window approach.

    Parameters:
    - df: DataFrame containing the dataset with features and target variable.
    - sliding_window: Number of rows to use for training in each sliding window.
    - lag_price_window: Number of previous days to use as features.

    Returns:
    - prediction_df: DataFrame containing predictions and actual values.
    """
    if DEBUG:
        print("Debug mode is ON. Detailed output will be printed.")

    # Validate input parameters
    if sliding_window <= lag_price_window:
        raise ValueError("Sliding window must be greater than to the lag price window.")

    training_sliding_window = sliding_window + 1  # +1 to include the next row as the test set

    # Calculate number of sliding window models to train in the dataset
    num_sliding_windows = len(df) - training_sliding_window
    if DEBUG:
        # Training for sliding windows and lag price window
        print(f"Training sliding window size: {training_sliding_window}, lag price window size: {lag_price_window}")
```

```
        print(f"Number of rows in the dataset: {len(df)}")

    print(f"Number of models to train: {num_sliding_windows}")

    # Initialize lists to store predictions, actuals, and timestamps
    predictions_list = []
    actuals_list = []
    timestamps_list = []

    model = preferred_model()

    for i in range(num_sliding_windows):
        if DEBUG:
            print(f"Processing sliding window {i + 1}/{num_sliding_windows}...")

        # Ensure we do not exceed the DataFrame length
        if i + training_sliding_window >= len(df):
            break  # Avoid index out of bounds

        # Extract the sliding window set, including the next row to use for prediction
        sliding_window_set = df.iloc[i : i + training_sliding_window]

        if DEBUG:
            print(f"Sliding window should have {training_sliding_window} rows, got {len(sliding_window_set)} rows.")
            print(f"Sliding window set:\n{sliding_window_set}")

        # Create feature matrix and target variable for training
        X_train, y_train = create_feature_matrix(sliding_window_set, lag_price_window)

        if DEBUG:
            print(f"Feature matrix shape: {X_train.shape}, Target variable shape: {y_train.shape}")
            print(f"Feature matrix:\n{X_train.head()}")
            print(f"Target variable:\n{y_train.head()}")

        # Split for training and prediction
        X_train_fit = X_train.iloc[:-1]
        y_train_fit = y_train.iloc[:-1]

        X_to_predict = X_train.iloc[-1:]
        y_to_predict = y_train.iloc[-1]

        if DEBUG:
            print(f"Training features shape: {X_train_fit.shape}, Training target shape: {y_train_fit.shape}")
```

```
            print(f"Features to predict shape: {X_to_predict.shape}, Target to predict: {y_to_predict}")

        model.fit(X_train_fit, y_train_fit)
        y_predicted = model.predict(X_to_predict)

        # Add a lower bounds to set extreme negative predictions to 0, assuming prices cannot be negative
        if y_predicted[0] < 0:
            y_predicted[0] = 0

        # Store results
        predictions_list.append(y_predicted[0])
        actuals_list.append(y_to_predict)
        if 'Datetime' in sliding_window_set.columns:
            timestamps_list.append(sliding_window_set.iloc[-1]['Datetime'])
        else:
            timestamps_list.append(i + training_sliding_window - 1)

    # Create final prediction DataFrame
    prediction_df = pd.DataFrame({
        'Timestamp': timestamps_list,
        'Predicted': predictions_list,
        'Actual': actuals_list
    })

    if DEBUG:
        print(f"Final prediction DataFrame:\n{prediction_df.head()}")

    return prediction_df
```

APPENDIX I: EXAMPLE OF THE METRIC CALCULATION PROCESS

```
# Define the Sliding Windows for the runs
sliding_windows = [7, 10, 15, 30, 90]  # of days to train on (matrix rows)
lag_price_windows = [1, 2, 4, 6]  # of previous days to use as features (matrix columns)
# lag_price_window must be less than sliding_window
percentiles = [99, 95, 85, 75]

# Initialize comprehensive results list
comprehensive_baseline_results = []

for sliding_window in sliding_windows:
    for lag_price_window in lag_price_windows:
        print(f"\nRunning regression training with sliding window: {sliding_window}, price feature window: {lag_price_window}")

        # Run the regression training
        prediction_df = regression_training_gt(df, sliding_window, lag_price_window, DEBUG)

        # Calculate overall metrics
        actuals_list = prediction_df['Actual'].values
        predictions_list = prediction_df['Predicted'].values
        timestamps_list = prediction_df['Timestamp'].values

        mse = mean_squared_error(actuals_list, predictions_list)
        mae = mean_absolute_error(actuals_list, predictions_list)
        r2 = r2_score(actuals_list, predictions_list)

        # Calculate prediction errors for filtering
        prediction_errors = np.abs(np.array(predictions_list) - np.array(actuals_list))

        # Calculate ES-like metric using prediction_errors
        # FIXED: Calculate worst 5% of errors across all predictions
        worst_overall_error_threshold = np.percentile(prediction_errors, 95)  # worst 5% of errors
        worst_overall_indices = prediction_errors >= worst_overall_error_threshold
        avg_worst_error = np.mean(prediction_errors[worst_overall_indices])

        # Add overall results (100% percentile)
```

```
        comprehensive_baseline_results.append({
            'sliding_window': sliding_window,
            'lag_price_window': lag_price_window,
            'percentile': 100,
            'data_points': len(predictions_list),
            'mse': mse,
            'mae': mae,
            'r2': r2,
            'worst_avg_error': avg_worst_error
        })

        for p in percentiles:
            # Filter by best predictions (lowest errors)
            best_error_threshold = np.percentile(prediction_errors, p)

            # Get indices of best predictions
            best_indices = prediction_errors <= best_error_threshold

            filtered_actuals = np.array(actuals_list)[best_indices]
            filtered_predictions = np.array(predictions_list)[best_indices]
            filtered_errors = prediction_errors[best_indices]

            if len(filtered_actuals) > 0:
                mse_p = mean_squared_error(filtered_actuals, filtered_predictions)
                mae_p = mean_absolute_error(filtered_actuals, filtered_predictions)
                r2_p = r2_score(filtered_actuals, filtered_predictions)

                # FIXED: Calculate worst 5% WITHIN the filtered subset
                worst_5_percent_threshold = np.percentile(filtered_errors, 95)
                worst_5_percent_indices = filtered_errors >= worst_5_percent_threshold
                worst_5_percent_errors = filtered_errors[worst_5_percent_indices]
                avg_worst_5_percent = np.mean(worst_5_percent_errors)

                comprehensive_baseline_results.append({
                    'sliding_window': sliding_window,
                    'lag_price_window': lag_price_window,
                    'percentile': p,
                    'data_points': len(filtered_predictions),
                    'mse': mse_p,
                    'mae': mae_p,
                    'r2': r2_p,
```

```
                        'worst_avg_error': avg_worst_5_percent  # FIXED: Now actually the average of worst 5%
                    })
                else:
                    # Add entry for no data available
                    comprehensive_baseline_results.append({
                        'sliding_window': sliding_window,
                        'lag_price_window': lag_price_window,
                        'percentile': p,
                        'data_points': 0,
                        'mse': np.nan,
                        'mae': np.nan,
                        'r2': np.nan,
                        'worst_avg_error': np.nan
                    })
```